

Armed Mobs 指令 / 配置 / 第三方内容参考

Minecraft 1.20.1 · Forge 47 · TaCZ 1.1.7+ · modId tarkovscav

命令 · 配置键 · AI 档位 · 数据包与第三方内容接口

项	值
显示名 / 内部 id	Armed Mobs（武装暴徒） / tarkovscav（不变，存档与配置兼容）
构建产物	armedmobs-0.1.0-all.jar
主命令根 / 别名	/armedmobs ... / /tarkovscav ...（同一棵树注册两次）
权限	服务端命令需权限等级 2；客户端命令无权限要求
配置文件	config/tarkovscav-common.toml
本文依据	源码逐条核对：ModCommands / ClientCommands / Config / AiProfile / registry/* / world/* / data/tarkovscav/**
校验方式	node tools/selftest_wiki_doc.js（少一条命令或一个配置键即报红）
文档版本	2026-09-26

说明：所有命令、参数、默认值与键名都逐条取自源码。凡无法从源码确证的内容，正文中一律标注（未核对）而不做猜测；第 9.1 节汇总了这些条目，第 9.2 节记录了本文撰写时发现并已修复的文档漂移。

目录

(Word 打开时按住 Ctrl 点击条目跳转；页码由 Word 自动生成)

目录.....	2
1. 怎么用这份表.....	5
1.1 版本 / id / 命令根.....	5
2. 服务端命令总表 (/armedmobs ...)	6
3. 客户端命令总表 (/armedmobs client ... + /armedmobs test killfeed)	10
3.1 client gunpose / client villagerpose 的语义轴约定 (源码注释确认)	11
4. 单位与 AI 档位对照表.....	12
4.1 九个实体 → 四个智力档 (来自 AiProfile#tierFor, 顺序承重: 先判子类, 未识别的落到 SCAV)	12
4.2 四档行为数值表 (出厂默认, 源码 Config.AiSettings)	12
4.3 枪械档 [tiers.<gun>] (ScavTier, 与 AI 档是两套不同的档位)	13
5. 配置键速查表 (按配置段)	14
5.1 [spawn]	14
5.2 [guns]	14
5.3 [combat]	15
5.4 [ai] (门行为 + 总开关)	15
5.5 [ai.coordination] (小队协同; 仅 TROOP / ELITE 参与)	16
5.6 [tactics] (掩体 / 压制 / 推进 / 脱离接触)	16
5.7 [tiers.<pistol shotgun rifle sniper>]	17
5.8 [voice]	17
5.9 [faction]	18
5.10 [alert] (阵营情报共享: 只报方向与距离带, 绝不报坐标)	18
5.11 [rack] (武器放置台)	18
5.12 [accuracy] (精度档 / warm-up)	19
5.13 [mods] (随机配件池「改枪王」)	19
5.14 [sniper]	19
5.15 [client] (模型 / 挂枪 / 姿势 / 歪头)	20
5.16 [client.headAccessories]	22

5.17 [killFeed]	23
5.18 [troops] (USEC 村民 / BEAR 掠夺者 / 优质村民 / 优质掠夺者)	23
5.19 [ricochet] (硬目标: 铁傀儡等)	24
5.20 [grenades] (投掷物总段)	24
5.21	
[grenades.frag] · [grenades.he] · [grenades.smoke] · [grenades.flash] · [grenades.flash Short]	25
5.22 [grenades.mob] (怪投雷)	25
5.23 [ai.scav] / [ai.sniper] / [ai.troop] / [ai.elite]	26
5.24 [command] (指挥系统: 三件阵营道具 + 信号棒 + 信号点方块)	26
5.25 [garrison] (一次性城市驻军 / 按楼分配阵营)	27
5.26 [shield] (VANT 防弹盾牌: 正面挡枪弹, 手雷照打)	28
5.27 [ladder] (爬梯: 九个武装单位自己上下梯子)	28
5.28 [capture] (主世界城市争夺战: 兵力池 + 血条; 废土无效)	29
6. 常用调参配方	30
6.1 让 AI 更凶 / 更快	30
6.2 让 AI 更笨 / 更慢	30
6.3 关掉门行为	31
6.4 关掉手雷	31
6.5 调试 AI (看日志 / 看状态)	31
6.6 测试用召唤	31
7. 第三方内容接口	33
7.1 实体类型 id (在本模组基础上做内容时引用)	33
7.2 本模组读取的实体类型标签 (数据包覆盖路径)	33
7.3 结构: tarkovscav:city 结构标签 + structures/buildings/ 目录	34
7.3.2 城区箱子的 TaCZ 注入 (global loot modifier, 2026-09-26)	36
7.3.1 部署信标 tarkovscav:deployment_beacon (进入城市废土维度的主要方式)	37
7.4 自定义结构运行管线: tarkovscav/city/*.nbt + city import reload place structures	38
7.5 自定义语音包 (voice_clips.txt / sounds.json)	38
7.6 枪械池 / TaCZ 集成键	39
7.7 其它扩展点	40

8. 数据 / 文件位置与日志标记41

 8.1 文件位置41

 8.2 日志标记（测试者按标记搜索）41

9. 未核对项与源码/README 不一致43

 9.1 （未核对）43

 9.2 撰写本文时发现并已修复的文档漂移（2026-09-25）43

本文面向三类读者：**服主**（用命令驱动、用配置调参）、**测试者**（复现战斗、读日志判定）、**第三方内容作者**（数据包、自定义结构、语音包、枪械池）。

所有命令、参数、默认值与键名都逐条取自源码（`command/ModCommands.java`、`command/ClientCommands.java`、`Config.java`、`gun/AiProfile.java`、`registry/*`、`world/*`、`src/main/resources/data/tarkovscav/**`）。凡本文无法从源码确证的，一律标注（**未核对**），不做猜测。

1. 怎么用这份表

- 1. **命令**：第 2 节是服务端命令，第 3 节是客户端命令。表里的「参数」列写明了取值范围（写 **[x]** 表示可省略），照抄「示例」即可。
- 2. **调参**：第 5 节按配置段列出**每一个**配置键及其出厂默认值。想快速达成某个效果，直接看第 6 节的「配方」。
- 3. **做内容**：第 7 节列出所有扩展点（实体、实体标签、结构、语音、枪械池）与最小可用示例；第 8 节列出运行时文件与日志标记的位置。

1.1 版本 / id / 命令根

项	值
modId（内部 id，不变）	tarkovscav
显示名 / display name	Armed Mobs（武装暴徒）
主命令根（推荐书写）	/armedmobs ...
兼容别名命令根（同一棵树）	/tarkovscav ...
服务端命令权限	需要权限等级 2 （ <code>source.hasPermission(2)</code> ）；在单人「允许作弊」的世界里即拥有
客户端命令权限	不需要 服务端权限；在单人世界和任何服务器上都能用（它们在客户端注册，见 <code>ClientCommands</code> ）
配置文件	<code>config/tarkovscav-common.toml</code> （Forge COMMON 类型，服务端与客户端各读一份）

`/armedmobs` 与 `/tarkovscav` 是**同一棵树注册两次**（`tree(rootLiteral)`），所以两者的子命令、参数、权限完全一致。本文表格统一以 `/armedmobs` 书写；把根换成 `/tarkovscav` 同样有效。

客户端命令里有一条挂在根下：`/armedmobs test killfeed`（它属于客户端树，不是服务端的 `/armedmobs test fight/watch/...`）。两者同名但来源不同：`test fight/watch/stall/sound/rack/mods/sniper/grenade` 由 `ModCommands` 注册（需要权限 2），`test killfeed` 由 `ClientCommands` 注册（客户端、无权限要求）。

2. 服务端命令总表 (/armedmobs ...)

全部叶子命令都需要权限等级 2。返回值即命令返回码（1 = 成功，0 = 被拒绝 / 参数错误，部分命令返回条目数）。

命令	参数（含取值范围）	作用	示例
<code>city add <name> [radius] [pos]</code>	name : 单词（string word，无空格）； radius : 整数 4..512，缺省 64； pos : 方块坐标，缺省=命令来源所在位置	把一个立方体范围标记为「城市区域」，追加/覆盖同名条目写入 <code>spawn.cityRegions</code> 并立即 <code>save</code> ，同时写 <code>[city]</code> 日志	<code>/armedmobs city add downtown 48 100 -60 100</code>
<code>city remove <name></code>	name : 单词	从 <code>spawn.cityRegions</code> 删除同名区域并保存；找不到则失败返回 0	<code>/armedmobs city remove downtown</code>
<code>city list</code>	无	打印 <code>cityRegions</code> 条目数、每条内容，以及 <code>cityStructureIds</code> / <code>cityStructureTags</code> 的当前值	<code>/armedmobs city list</code>
<code>city import <name></code>	name : 单词	读取 <code><world>/generated/minecraft/structures/<name>.nbt</code> ，校验（压缩 NBT、size/palette/blocks 合法、每格 palette 索引存在、单轴 ≤ 512、方块数 ≤ 250000）后复制到 <code><gameDir>/tarkovscav/city/<name>.nbt</code> 并注册进运行时池	<code>/armedmobs city import mycity</code>
<code>city reload</code>	无	重新扫描 <code><gameDir>/tarkovscav/city/*.nbt</code> 并全部注册；返回加载到的名字数量	<code>/armedmobs city reload</code>
<code>city place <name> [pos] [rotation] [mirror]</code>	name : 单词； pos : 方块坐标，缺省=自身位置； rotation : 整数 0..270（90/180/270 顺时针/半圈/逆时针，其余按 0 处理）； mirror : 单词， <code>left_right leftright x</code> → 左右镜像， <code>front_back frontback z</code> → 前后镜像，其它值 → 不镜像（缺省 none）	放置一个运行时结构实例，记录其包围盒（供刷怪门判定），并打印 <code>ACCEPT/REJECT</code> 结论与包围盒	<code>/armedmobs city place mycity ~ ~ ~ 90 left_right</code>
<code>city structures</code>	无	列出 jar 内置结构 id、运行时池（ <code>tarkovscav/city/</code> ）条目、本次会话已放置实例，并说明「运行时 nbt 不能变成 worldgen 结构」的限制	<code>/armedmobs city structures</code>
<code>city district [seed] [grid]</code>	seed : 长整数，缺省=世界随机； grid : 整数 1..7，缺省 3（3 = 48×48 格街道）	用与 worldgen 结构相同的 NBT 与 template pool 在当前位置即时拼一个城区（建筑概率 0.7、装饰概率 0.5），打印 <code>seed/grid/streets/buildings/decor/bounds</code> ，同 seed 可复现	<code>/armedmobs city district 123456789 5</code>
<code>city test [pos]</code>	pos : 方块坐标，缺省=自身位置	用真正的 CityGate 判定该点是否城市区域，打印 <code>ACCEPT/REJECT</code> 与原因、区块结构诊断，并写 <code>[spawngate]</code> 日志；命中返回 1，未命中返回 0	<code>/armedmobs city test 100 -60 100</code>
<code>spawn <type> [pos]</code>	type : 单词。可取值见下方「可召唤类型」； pos : 方块坐标，缺省=自身位置	穿过刷怪门召唤一只怪：不在城市区域会被拒绝并说明原因。成功时打印档位，狙击手/部队还会追加枪械、精度档、护甲等级、语音池等摘要；写 <code>[spawngate]</code> 日志	<code>/armedmobs spawn sniper 100 -60 100</code>

命令	参数（含取值范围）	作用	示例
<code>test fight</code> [distance] [pos]	distance: 双精度 4.0..64.0, 缺省 16.0; pos: 缺省=自身位置	搭一场可复现的对抗: 一只 <code>scav</code> 对一只无 AI 练习假人（带 <code>tarkovscav_dummy</code> 记分板的僵尸，放在东侧 distance 格），假人被设为耐久血量	<code>/armedmobs test fight 24</code>
<code>test watch</code> [seconds] [distance] [pos]	seconds: 整数 2..600, 缺省 20; distance: 4.0..64.0, 缺省 16.0; pos 可选	同上搭建，窗口结束后在日志断言「移动 ≥ 1.0 格且开火 ≥ 3 发」，打印 [test] WATCH PASS/FAIL ... moved= shots= dummyDamage= stalls= state=; 防卡死回归闸门	<code>/armedmobs test watch 30 16</code>
<code>test stall</code> [seconds] [distance] [pos]	同上	同上，但窗口内每一枪都被拒绝（ <code>FORGE_EVENT_CANCEL</code> ，不调用 TaCZ），断言反卡死看门狗至少触发 1 次（ <code>escapes >= 1</code> ）	<code>/armedmobs test stall 20</code>
<code>test sound</code> [name] [pitch]	name: 单词，缺省 all。可取值: all、idle、chatter、contact、taunt、grenade、mark、death、家族名（ <code>usec bear elite</code> ）、家族池名（如 <code>usec_contact</code> ）、实体名（如 <code>usec_villager</code> ）、单个 clip 名（如 <code>contact_1</code> 、 <code>usec_contact_3</code> 、 <code>grenade_land</code> ）; pitch: 双精度 0.1..2.0, 缺省不传则在整个音高带上扫描	在命令位置逐条播放语音（每 1.5 s 一条）。带 pitch 时全部用该音高; 不带时按 <code>voice.pitchMin..pitchMax</code> 均分，一次听完整个音域。每行都写 [sound] 日志	<code>/armedmobs test sound usec_contact</code>
<code>test rack</code>	无	报告 16 格内最近的武器放置台: 内容物、 <code>kind=normal creative、infinite</code> 、朝向、军械台开关、兵装分类、取用冷却、可接受清单，以及「谁会来拿、拿走会变成什么」和掉落吸收扫描的当前结果; 写 [rack] 日志	<code>/armedmobs test rack</code>
<code>test mods</code>	无	报告 32 格内每只持枪单位的手上枪械与配件，以及 [mods] 池状态、TaCZ 配件索引是否为空、脚本枪排除规则与保留清单; 写 [mods] 日志	<code>/armedmobs test mods</code>
<code>test sniper</code>	无	报告 64 格内每只狙击手（ <code>sniper_pillager / sniper_villager</code> ）的蹲点状态、档位、精度档与上限、实际发的枪、弹匣容量、 <code>FOLLOW_RANGE</code> 、到玩家距离、护甲等级、语音池; 无狙击手则失败返回 0; 写 [sniper] 日志	<code>/armedmobs test sniper</code>
<code>test grenade</code> [type]	type: 单词，缺省 frag。取值: frag、he、smoke、flash、flash_short	在视线方向求解一条投掷弧（与 AI 用同一套弹道求解器），打印落点、半径、发射俯仰、弧线是否通畅、引信; 打印 1/2/4/6/8 格的期望伤害表; 列出半径内生物与墙体遮挡; 若命令来源是玩家则真的扔一颗。 <code>grenades.enabled = false</code> 时拒绝	<code>/armedmobs test grenade flash</code>
<code>cover</code>	无	取 32 格内最近一只持枪单位，按其当前目标打印所有可见掩体候选点（含 <code>hidden/open</code> 判定、数量、搜索半径与缓存 tick），并写 [cover] 日志; 无目标时提示先跑 <code>test fight</code>	<code>/armedmobs cover</code>

命令	参数（含取值范围）	作用	示例
<code>gunpool [tier]</code>	<code>tier</code> : 可省略；字面量为枪械档 id <code>pistol</code> <code>shotgun</code> <code>rifle</code> <code>sniper</code> ；缺省按 <code>rifle</code>	打印该档当前可被发放的全部 TaCZ 枪 id 及其 type，并报告被 <code>gunBlacklist/gunWhitelist/excludedGunTypes</code> 与脚本枪规则过滤掉的数量；池为空时红字提示检查 TaCZ 枪包	<code>/armedmobs gunpool sniper</code>
<code>debug</code>	无	打印 32 格内每只持枪单位的完整状态机报告 (<code>GunBrain#debugSummary</code>) + 阵营/叛变/警戒网络/精度档/护甲等级/语音池/当前目标（是否 <code>VILLAGE-HOSTILE</code> ）；写 <code>[debug]</code> 日志	<code>/armedmobs debug</code>
<code>dimension [name]</code>	<code>name</code> : 单词，可省略。省略 = <code>tarkovscav:urban_wasteland</code> ；带命名空间按原样解析，不带则补 <code>tarkovscav</code> 。自动补全列出本服务器所有已加载维度	把执行者（必须是玩家）传送到城市废土维度（或指定维度），落点与部署信标完全同一套安全落地代码（ <code>MOTION_BLOCKING</code> 高度图定 Y、悬空时铺 5×5 石砖平台 + 火把），并打印落点坐标；写 <code>[wasteland]</code> 日志。非玩家来源、未知维度都会干净拒绝并说明	<code>/armedmobs dimension、 /armedmobs dimension minecraft:the_nether</code>
<code>fillwater [radius] [block]</code>	<code>radius</code> : 整数 1..128，可省略（默认 48）。 <code>block</code> : 字符串，可省略（默认 <code>minecraft:stone</code> ），不带 <code>minecraft</code> : 都认； <code>minecraft:air</code> = 抽干	以执行者为中心扫一个立方体，把 <code>minecraft:water</code> 替换成 <code>block</code> （1.20.1 没有独立的 <code>flowing_water</code> 方块，流动与静止都是带 0.15 液面属性的 <code>minecraft:water</code> ，所以一条判定就够）。含水方块（楼梯/栅栏/台阶等）只计数、不替换——替换它们等于把方块本身删掉；未加载的区块跳过并在结果里标注；替换位置落在扫描边界上的数量单独报告并提示「外面的水会流回来，再来一次或把半径放大」。打印 <code>[fillwater]</code> 行：维度、目标方块、包围盒、扫描数、替换数、含水跳过数、边界水数。未知方块名干净拒绝返回 0	<code>/armedmobs fillwater、 /armedmobs fillwater 96、 /armedmobs fillwater 128 minecraft:air</code>
<code>marks</code>	无	列出当前维度的全部存活标记：字母、坐标、来源（ <code>tool</code> / <code>stick</code> / <code>point</code> ）、剩余时间（ <code>permanent</code> 或秒数），并打印本维度的上限 <code>command.maxMarks</code> 与影响半径 <code>command.radius</code> ；返回 1	<code>/armedmobs marks</code>
<code>marks remove <letter></code>	<code>letter</code> : 单词（大小写不敏感，会先被 <code>CommandMark#sanitiseLetter</code> 归一化）	删除该字母的标记（不分来源；信号点方块的标记也能这样删）。成功后提示「指向它的命令会被丢弃」；该维度没有这个字母则失败返回 0	<code>/armedmobs marks remove B</code>
<code>marks clear</code>	无	清空当前维度的全部标记，打印清除数量；本来就没有标记时返回 0	<code>/armedmobs marks clear</code>
<code>garrison</code>	无	打印一次性城市驻军的生效配置（是否启用、每城小队数是「固定值」还是「按尺寸公式」并给出公式、小队人数区间、精英队长概率、触发半径、检查间隔、按楼阵营的四个概率/开关、TROOP/ELITE id 清单），再逐条列出账本：已放置驻军的维度、城市标识、小队数、单位数、放置时的 game tick；最后列出每个已决定阵营的城市及其每一栋楼的阵营。返回 1	<code>/armedmobs garrison</code>

命令	参数（含取值范围）	作用	示例
<code>city faction</code>	无	只打印阵营账本：每个已决定阵营的城市一行（维度、城市标识、 dominant= 、可选 override= 、 spawners=rewritten pending ），其下每栋楼一行（<序号:楼名> -> 阵营，覆写时另注 override 与原掷骰）。返回 1	<code>/armedmobs city faction</code>
<code>city faction <key> <faction></code>	key : 单词，账本里的城市标识；在末尾加 #<buildingId> 可只改一栋楼。 faction : <code>village</code> / <code>illager</code> / <code>auto</code>	覆写指定城市（或指定楼）的阵营，写入账本并清除「刷怪笼已改写」标记，因此下一次驻军触发会按新阵营重写刷怪笼。 auto 清除覆写、回落到当初记录的掷骰。未知城市/楼、未知阵营返回 0	<code>/armedmobs city faction structure/tarkovscav:city_small/10,-4,10 illager</code>
<code>city faction <faction> [pos]</code>	faction : 同上； pos : 可选坐标，缺省用命令执行者所在位置	解析该位置（默认取最近的城市盒）所在的城市，覆写其阵营并立即生效：立刻按楼重写已加载区块的刷怪笼、并补刷尚未放置的驻军（覆写不会改变已经站好的驻军成员）。半径内没有城市（或区块未加载）时返回 0。控制台可用 <code>/execute positioned 0 70 0 run armedmobs city faction village</code>	<code>/execute positioned 0 70 0 run armedmobs city faction village</code>
<code>capture</code>	无	先打印生效配置（开关、血条开关、池上下限、每栋楼增量、每次死亡扣减、是否只算玩家击杀、血条隐藏延迟），再逐城列出账本里的池：维度、城市标识、每个阵营的 当前/上限 、是否已 captured （占领）。返回 1	<code>/armedmobs capture</code>
<code>capture reset [at [pos] <城市键>]</code>	不写 = 最近的城市； at [pos] 同义（可给坐标）； <城市键> = 账本里的城市标识	按账本里记录的建筑数与该城当前阵营重建该城全部兵力池，并清掉 captured 标记，于是这座城重新进入争夺（已经站好的驻军成员不会被清除）。找不到城市返回 0。只主世界有效	<code>/armedmobs capture reset</code> <code>/armedmobs capture reset at 0 70 0</code> <code>/armedmobs capture reset structure/tarkovscav:city_small/10,-4,10</code>
<code>capture set <faction> <value> [pos]</code>	faction : <code>village</code> / <code>illager</code> ； value : 整数 0..400； pos : 可选坐标，缺省用执行者位置	把最近的城里该阵营的池直接设成 value （0 = 立刻判占领并写 <code>[capture] ... captured by ...</code> 日志），用于不杀一百个怪就测结局与血条。设成 0 时同样只写一次占领日志。找不到城市或该城没有这个阵营的池返回 0	<code>/armedmobs capture set illager 0</code> <code>/armedmobs capture set village 5 120 70 -40</code>

可召唤类型（**spawn <type>**）：`tarkovscav:scav`、`tarkovscav:gunner_pillager`、`tarkovscav:gunner_villager`、`tarkovscav:sniper_pillager`、`tarkovscav:sniper_villager`、`tarkovscav:usec_villager`、`tarkovscav:bear_pillager`、`tarkovscav:elite_villager`、`tarkovscav:elite_pillager`。

- 短名可以省略命名空间；**sniper** 是 `sniper_pillager` 的简写别名（`typeName.equalsIgnoreCase("sniper")` → `sniper_pillager`）。
- 也接受完整 id 形式（含 **:** 的 `namespace:path`），但必须是上表九个之一，否则报错返回 0。
- 错误提示串里列出的名字漏写了 `sniper_pillager`，但它是合法的——以本表的九个 id 为准。

3. 客户端命令总表 (/armedmobs client ... + /armedmobs test killfeed)

下面全部由 **ClientCommands** 注册，跑在你自己的客户端上，**不需要服务端权限**（单人世界、任意服务器均可用）。所有写配置的命令都会立刻 **save** 到 **config/tarkovscav-common.toml**，并在下一帧生效。

命令	参数语法（真实解析器）	作用	示例
client reload	无	从磁盘重读 config/tarkovscav-common.toml (Forge 的 ConfigTracker#loadConfigs(COMMON) ，注意它会把 所有 mod 的 COMMON 配置一并重读)，并使隐藏骨骼、头部配件、头部俯仰、挂枪变换、配件池、枪池、脚本枪缓存、跳弹缓存在下一帧重算	/armedmobs client reload
client state	无	打印当前真正生效的值：合并后的隐藏骨骼集合（每根带原因）、头配件清单、 headRestPitchDegrees 与生效状态、锚点/离手锚点/锚点模式/显示上下文、离手渲染与双手支撑、步枪与手枪的挂枪变换、模型分层与渲染类型、两个模型缩放（ rig 与 villager ）、武装村民的枪与四姿势、语音音量与家族倍率、姿势写入者统计、玩家歪头状态	/armedmobs client state
client hide <bone>	bone : 单词（骨骼名）	把一根骨骼加入 client.headAccessories.extraHiddenBones （诊断用二分法：一次藏一根，看问题何时消失）；已存在则失败返回 0	/armedmobs client hide Hat2
client show <bone>	bone : 单词	从 extraHiddenBones 移除该骨骼；不存在则失败返回 0	/armedmobs client show Hat2
client gunpose [args...]	贪心字符串，空格分隔，任意顺序，全部可省略： family = class = rifle pistol ; hand = main offhand ; pitch = rx =、 yaw = ry =、 roll = rz =（度，按 X→Y→Z 应用）； x =、 y =、 z =（设置偏移，方块）； scale =（缩放）； forward = barrel =、 back =、 right =、 left =、 up =、 down =（沿锚点轴同名轴的增量微调，可反复点按）； context =（任意 ItemDisplayContext 名，大写）； mode = normalisedHand locatorAnimated ; reset default	实时调枪械变换：应用到下一帧、打印可粘贴的 toml 行并写盘。 x/y/z 是「设置」，方向名是「增量」。 reset 回到出厂基线（非零）：步枪/手枪 rot [0,0,0] 、 offset [0,0,-0.7] 、 scale 1.0 ，离手归零，并恢复 gunMountDisplayContext=THIRD_PERSON_RIGHT_HAND 、 gunAnchorMode=normalisedHand	/armedmobs client gunpose yaw=15 forward=0.05
client scale [args...]	贪心字符串： 裸 scale （无参）→ 打印当前值； scale <value> 、 scale up 、 scale down 、 scale up <step> 、 scale down <step> ；带前缀 villager villagers 时改的是村民家族的键： scale villager <value> 、 scale villager up [step] 等。 取值范围 0.3..2.0 （超出自动夹取并提示）， step 缺省 0.05 ；非数字被拒绝	实时调模型大小：不带 villager 写 client.renderScale (Bedrock rig : scav + gecko 掠夺者，基线 0.77)，带 villager 写 client.villagerRenderScale （村民家族， 1.0 = 原版村民大小）	/armedmobs client scale villager 1.05

命令	参数语法（真实解析器）	作用	示例
<code>client villagerpose [args...]</code>	贪心字符串，空格分隔，任意顺序，全部可省略： <code>pitch=</code> 、 <code>yaw=</code> 、 <code>roll=</code> 、 <code>x=</code> 、 <code>y=</code> 、 <code>z=</code> 、 <code>scale=</code> ；语义轴 <code>forward=</code> （-Z 前进）、 <code>back=</code> 、 <code>right=</code> 、 <code>left=</code> 、 <code>up=</code> 、 <code>down=</code> ； <code>anchor=arms anchor=body</code> ；四姿势手臂俯仰偏移 <code>aim=</code> 、 <code>hold=</code> 、 <code>reload=</code> 、 <code>hunker=</code> ； <code>hideGun=on off true false</code> ；闲置姿势专用 <code>idlePitch=</code> 、 <code>idleYaw=</code> 、 <code>idleRoll=</code> ；换弹姿势专用 <code>reloadPitch=</code> 、 <code>reloadYaw=</code> 、 <code>reloadRoll=</code> ；撤退姿势专用 <code>hunkerPitch=</code> 、 <code>hunkerYaw=</code> 、 <code>hunkerRoll=</code> ；每姿势位置增量 <code>idleX/Y/Z=</code> 、 <code>reloadX/Y/Z=</code> 、 <code>hunkerX/Y/Z=</code> ； <code>reset</code> （从出厂 <code>DEFAULT_*</code> 常量恢复全部）	实时调武装村民的枪与姿势（写 <code>client.gunnerVillager*</code> 等键），打印可粘贴 toml 行并保存。先定手臂角再调枪位置（枪挂在 <code>arms</code> 上，手臂一动枪就跟着动）	<code>/armedmobs client villagerpose aim=-57 idleX=0.05 hideGun=on</code>
<code>client pose [args...]</code>	贪心字符串（大小写不敏感）： 裸 <code>pose</code> → 打印当前值； <code>pose auto code clips</code> （写 <code>client.poseSource</code> ， <code>PoseSource</code> 枚举的三个值）； <code>pose molang</code> → 在 <code>pitch→all→off→pitch</code> 之间循环； <code>pose molang on true all yaw</code> → <code>ALL</code> ； <code>pose molang pitch pitchonly</code> → <code>PITCH</code> ； <code>pose molang off false none</code> → <code>OFF</code> ； <code>pose torso <0..1></code> 或 <code>pose torso=<0..1></code> （写 <code>client.torsoYawShare</code> ，超界夹取）	一键 A/B：谁拥有瞄准姿势骨骼（ <code>poseSource</code> ）、是否喂给 rig 自己的 Molang 瞄准变量（ <code>molangVariables</code> ）、胸腔承担多少看向偏移（ <code>torsoYawShare</code> ，头部总是承担其余部分，所以瞄准落点不变）	<code>/armedmobs client pose molang all</code>
<code>/armedmobs test killfeed [killer] [victim] [weapon]</code>	<code>killer</code> ：字符串，缺省 <code>Ge_SiLa</code> ； <code>victim</code> ：字符串，缺省 <code>Armed Mobs</code> ； <code>weapon</code> ：字符串，缺省 <code>fists</code> 。取值：物品 id（如 <code>minecraft:stone_sword</code> → <code>ITEM</code> ）、类别 id <code>item fists explosion fall drowning fire magic environment unknown</code> ，或省略	直接在 HUD 上压入一条击杀播报，用来肉眼确认布局、配色、截断与行数上限（不会杀死任何东西）；打印当前 <code>killfeed</code> 生效值	<code>/armedmobs test killfeed Steve Herobrine minecraft:bow</code>

3.1 `client gunpose` / `client villagerpose` 的语义轴约定（源码注释确认）

在默认 `gunAnchorMode = normalisedHand` 下，锚点帧的轴按「它指向的方向」命名：

名字	轴	备注
<code>forward</code> / <code>back</code>	-Z / +Z	枪口轴；用户实测 <code>z=-0.7</code> 让步枪前移，故 -Z 朝枪口
<code>right</code> / <code>left</code>	+X / -X	角色侧向（该帧基向量 +x 映射到模型 -X）
<code>up</code> / <code>down</code>	-Y / +Y	帧基向量 +y 映射到模型 -Y

`x=`/`y=`/`z=` 是绝对设置；`forward=`/`up=` 等是增量（可反复点按累加）。`locatorAnimated` 模式下锚点帧被旋转，上面的方向名指向别处（只有 `normalisedHand` 模式成立）。

4. 单位与 AI 档位对照表

4.1 九个实体 → 四个智力档（来自 AiProfile#tierFor，顺序承重：先判子类，未识别的落到 SCAV）

<!-- ENTITY_TIER_TABLE_START -->

档位 ([ai.<tier>])	实体 id	实现类
SCAV	tarkovscav:scav · tarkovscav:gunner_pillager · tarkovscav:gunner_villager	ScavEntity / GunnerPillagerEntity / GunnerVillagerEntity（SCAV 是兜底分支）
SNIPER	tarkovscav:sniper_pillager · tarkovscav:sniper_villager	SniperPillagerEntity / SniperVillagerEntity
TROOP	tarkovscav:usec_villager · tarkovscav:bear_pillager	UsecVillagerEntity / BearPillagerEntity
ELITE	tarkovscav:elite_villager · tarkovscav:elite_pillager	EliteVillagerEntity / ElitePillagerEntity

<!-- ENTITY_TIER_TABLE_END -->

映射规则：ELITE 分支先判 EliteVillagerEntity/ElitePillagerEntity → TROOP 判

UsecVillagerEntity/BearPillagerEntity → SNIPER 判

SniperVillagerEntity/SniperPillagerEntity → 其余落到 SCAV。新增第十种单位只需在 tierFor 加一行。

4.2 四档行为数值表（出厂默认，源码 Config.AiSettings）

[ai] enabled = false 时整层关闭，所有方法返回全局键原值（即加此层之前的旧行为）。带 *Scale 的键是乘数，乘在对应的全局键上；其余是绝对值。

<!-- TIER_BEHAVIOUR_TABLE_START -->

键 ([ai.<tier>])	SCAV	SNIPER	TROOP	ELITE	作用	取值范围
reactionMinTicks	12	16	6	4	发现目标后的反应延迟下限（tick；每次获得目标重掷，绝对值，覆盖 [combat] reactionTicks）	0..200
reactionMaxTicks	24	30	10	8	反应延迟上限（等于下限则无抖动）	0.400
accuracyScale	0.80	1.0	1.0	1.05	乘到枪械档精度上（再由精度档封顶）	0.0..2.0
coverChance	0.15	1.0	0.90	0.75	每次「用掩体」的决定真正去找掩体的概率（换弹/撤退不受此限）	0.0..1.0
coverRadiusScale	0.45	1.0	1.0	0.85	乘 tactics.coverSearchRadius	0.1..3.0
coverCacheScale	1.5	1.0	1.0	0.75	乘 tactics.coverCacheTicks	0.1..5.0
suppressChanceScale	0.0	0.0	1.6	1.0	乘 tactics.suppressChance（结果夹到 0..1）	0.0..3.0
suppressTicksScale	0.0	0.0	1.5	1.0	乘 tactics.suppressTicks	0.0..5.0
suppressAccuracyScale	1.0	1.0	0.80	1.0	乘 tactics.suppressAccuracyMultiplier	0.05..2.0
suppressBurstScale	0.5	1.0	1.2	1.0	乘 tactics.suppressBurstMultiplier	0.1..5.0
advanceCoverScale	0.0	1.0	0.6	1.5	乘 tactics.advanceCoverStep；0 = 直线推进不用掩体	0.0..5.0
coverSeekSpeedScale	1.0	1.0	1.0	1.15	乘 tactics.coverSeekSpeedModifier	0.5..3.0
repositionScale	1.0	1.0	0.6	0.5	乘 combat.repositionTicks	0.1..3.0
retreatHealthScale	1.0	0.6	1.0	0.7	乘 combat.retreatHealthFraction	0.1..2.0
engageRangeScale	1.0	1.0	1.0	0.6	乘枪械档 engageRange（精英贴得更近才开火）	0.2..2.0

键 ([ai.<tier>])	SCAV	SNIPER	TROOP	ELITE	作用	取值范围
holdPost	false	true	false	false	true = 目标超出射程时原地重新选位而不是推进（只有狙击档为 true）	布尔
minHitChance	0.0	0.5	0.0	0.0	估计命中率低于它就不开这一梭；0 = 关闭（只有狙击档为 0.5，用稳态锥估计，不含 warm-up）	0.0..1.0
patienceTicks	0	100	0	0	为等待命中率门槛最多停留多久；0 = 无上限（只在 minHitChance > 0 时有意义）；连续 3 个窗口后仍不满足则直接开枪	0..600
coordination	false	false	true	true	是否参加小队层（集火/交替掩护/包抄/掩体独占）；还需 [ai.coordination] enabled	布尔
partialCoverBonus	250.0	250.0	250.0	250.0	狙击选点的「部分隐蔽或在目标上方 ≥1 格」额外分；0 = 关闭（四档同值）	0.0..1000.0
exposedBurstShots	0	1	-1	-1	目标暴露时一梭几发：-1 = 打空弹匣/打死为止，0 = 不覆盖（交回 [tiers.<gun>] burstShots），>0 = 正好这么多发	-1..200
exposedBurstCooldownTicks	-1	-1	0	0	暴露时梭间停顿：-1 = 不覆盖（用枪械档 burstCooldownTicks），0 = 完全不停；暴露规则需本键 >= 0 且上键 != 0 才接管扳机	-1..400
warmupShotsWhenExposed	-1	-1	0	0	暴露期间用哪个 warm-up 阈值：-1 = 用全局 accuracy.warmupShots(8)，0 = 首发即稳态，N = 前 N 发仍偏	-1..200
retreatHealthFraction	-1.0	-1.0	0.5	0.55	绝对血量比例，低于它脱离接触；-1 = 不覆盖（用 combat.retreatHealthFraction × retreatHealthScale）	-1.0..1.0
hurtRetreatChance	-1.0	-1.0	0.8	0.85	单次中弹直接转入 RETREAT 的概率；-1 = 用全局 [tactics] hurtRetreatChance	-1.0..1.0
retreatHoldTicks	0	60	80	60	脱离接触并进入掩体后至少待多久才可以重新交战；0 = 关闭（旧行为：40 tick 后重判）	0..600

<!-- TIER_BEHAVIOUR_TABLE_END -->

派生出的实际值（全局键 × 档位乘数，便于理解表意）举例：TROOP suppressChance = 0.6 × 1.6 = 0.96、suppressTicks = 60 × 1.5 = 90；ELITE advanceCoverStep = 3.0 × 1.5 = 4.5；SCAV coverSearchRadius = 14 × 0.45 ≈ 6。

4.3 枪械档 [tiers.<gun>]（ScavTier，与 AI 档是两套不同的档位）

键 ([tiers.<gun>])	pistol	shotgun	rifle	sniper	作用	取值范围
spawnWeight	40	25	30	8	该枪械档被抽中的相对权重；0 = 从世界移除该档	0..1000
health	20.0	24.0	26.0	22.0	该档最大生命值	1.0..1024.0
armor	0.0	2.0	4.0	1.0	该档护甲点（钻石胸甲为 8）	0.0..30.0
accuracy	0.45	0.55	0.6	0.85	0..1，消除多少瞄准误差（1.0 = 从不失手，但还会被 [accuracy] hardCeiling 封顶）	0.0..1.0
aimTicks	6	8	12	26	每梭第一发前额外瞄准 tick（叠加在 TaCZ 自身瞄准时间上）	0..400
burstShots	3	2	6	1	每梭发数；-1 = 打空弹匣或目标脱离视线	-1..200
burstCooldownTicks	16	30	20	45	梭间停顿 tick	0..400

ScavTier 与 TaCZ gun type 的对应：pistol 档 ← pistol、smg；shotgun 档 ← shotgun；rifle 档 ← rifle、mg；sniper 档 ← sniper。各档交战距离：pistol 18.0、shotgun 14.0、rifle 30.0、sniper 52.0 格。新增/自定义枪包按其 type 自动进池，无需改代码。

5. 配置键速查表（按配置段）

配置文件：`config/tarkovscav-common.toml`。下表逐键列出**每一个 define*** 注册点。每档键在 **5.23** (`[ai.<tier>]`，26 键 × 4 档) 与 **5.7** (`[tiers.<gun>]`，7 键 × 4 档)，共 132 个注册点。

5.1 [spawn]

键名	默认值	作用	备注/推荐范围
<code>spawn.scavCityOnly</code>	<code>true</code>	<code>true</code> = scav 只在城市区域内自然刷新；关闭即全局入侵	关掉会让暴徒遍布全世界
<code>spawn.gunnerPillagerCityOnly</code>	<code>true</code>	持枪掠夺者只在城市区域自然刷新	同上
<code>spawn.gunnerVillagerCityOnly</code>	<code>true</code>	持枪村民只在城市区域自然刷新	同上
<code>spawn.gunnerVillagerNaturalSpawns</code>	<code>true</code>	<code>false</code> = 持枪村民永不出现在生物群系刷新器里（只能靠刷怪蛋 <code>//summon//armedmobs spawn</code> ）	自然权重在 <code>data/tarkovscav/forge/biome_modifier/add_scavs.json</code> （村民权重 2，scav 5）
<code>spawn.gateCommandSpawns</code>	<code>false</code>	<code>false</code> = <code>/summon</code> 与刷怪蛋无视城市门（便于手放测试）； <code>true</code> = 连它们也被门拒绝	只想测战斗时保持 <code>false</code>
<code>spawn.cityStructureIds</code>	<code>["tarkovscav:city_small", "tarkovscav:city_a", "tarkovscav:city_b", "tarkovscav:city_c", "tarkovscav:city_strongpoint"]</code>	让一个坐标算作城市的 结构 id 列表	可填别的模组 id，如 <code>somecitymod:downtown</code> 、 <code>minecraft:village_plains</code>
<code>spawn.cityStructureTags</code>	<code>["tarkovscav:city"]</code>	让一个坐标算作城市的 结构标签 列表	接别人城市模组最省事的钩子；可写 <code>#namespace:tag</code> 或裸名
<code>spawn.cityRegions</code>	<code>[]</code>	手工城市盒。格式： <code>[<name> <dimension> <x1> <y1> <z1> <x2> <y2> <z2>]</code> ，例 <code>"minecraft:overworld -120 40 -80 120 120 80"</code> ；带名字形式 <code>"downtown minecraft:overworld -120 40 -80 120 120 80"</code>	游戏内用 <code>/armedmobs city add</code> 写；名字省略时解析为 <code>region</code>
<code>spawn.cityRegionPadding</code>	<code>4</code>	结构周围多少格仍算城市（街道与外圈）。这是城区刷怪率的 线性旋钮 ： <code>24 -> 4</code> 让单城区可刷怪面积约 -52.7%（原 24）。注意 Forge 不会重写已存在的 <code>config/tarkovscav-common.toml</code> ，老存档会继续用 <code>24</code> 直到你手改	0.256
<code>spawn.logSpawnGate</code>	<code>true</code>	把每次刷怪门的接受/拒绝写进服务端日志	接线城市模组时开，正常游玩偏吵
<code>spawn.foundationDepth</code>	<code>5</code>	<code>/armedmobs city district</code> 放置每块地皮时保证的地基深度（向上补足）	0.16；与生成器 <code>--foundation N</code> 保持一致（内置件烘死为 5，街道 3，瓦砾 0）

5.2 [guns]

键名	默认值	作用	备注/推荐范围
<code>guns.gunBlacklist</code>	<code>["tacz:rpg7", "tacz:m320", "tacz:minigun"]</code>	永不发给怪的枪 id	默认排除爆炸物与转轮机枪
<code>guns.gunWhitelist</code>	<code>[]</code>	非空时 只允许 这些 id（黑名单仍然生效）	做「只有 AK」的整合包时用
<code>guns.excludedGunTypes</code>	<code>["rpg"]</code>	永不发放的 TaCZ gun type 值	火箭筒不是公平的怪物武器

键名	默认值	作用	备注/推荐范围
<code>guns.pistolClipTypes</code>	<code>["pistol"]</code>	使用模型 <code>*_pistol</code> 单手动画的 TaCZ gun type	想给 SMG 用单手姿势就把 <code>"smg"</code> 加进来 (YSM 只特判 <code>pistol</code> 与 <code>rpg</code>)
<code>guns.excludeScriptedGuns</code>	<code>true</code>	把带 Lua 脚本的枪挡在「脚本命名空间不可信」的怪之外	崩溃防护; 详见 README §5p
<code>guns.trustedScriptNamespaces</code>	<code>["tacz"]</code>	即使开了上一条仍允许到达怪的脚本命名空间	信任的枪包命名空间
<code>guns.scriptedGunRescanTicks</code>	<code>40</code>	怪每隔多少 tick 重新检查手上枪是否符合脚本枪规则	0..2400
<code>guns.respectDeclaredFireModes</code>	<code>false</code>	<code>true</code> = 让枪自己的数据文件决定发放弹匣的开火模式, 而不是强制全自动	影响 <code>api:getFireMode()==AUTO</code> 分支的脚本枪 (如 <code>hamster:win1894</code>)
<code>guns.gunDropChance</code>	<code>0.35</code>	死亡掉落所持 TaCZ 枪的概率	0.0..1.0
<code>guns.ammoDropChance</code>	<code>0.5</code>	死亡掉落备用弹药的概率	0.0..1.0
<code>guns.ammoItemStacks</code>	<code>3</code>	怪携带的匹配弹药堆数; 打完会脱离接触	1..27
<code>guns.manualReloadFallback</code>	<code>true</code>	当 TaCZ 自己的 <code>reload()</code> 对生物无动作时, 由本模组补上换弹 (仍走 TaCZ API、仍消耗真实弹药)	关掉可观察 TaCZ 原始行为
<code>guns.manualReloadTicks</code>	<code>45</code>	补位换弹耗时 (tick), 期间怪躲掩体并播放换弹姿势	5..400
<code>guns.reloadStallTicks</code>	<code>200</code>	TaCZ 报告「换弹中」多久后不再相信并脱离接触	20..2400; 这是唯一由 TaCZ 驱动的状态的上界

5.3 [combat]

键名	默认值	作用	备注/推荐范围
<code>combat.reactionTicks</code>	<code>12</code>	发现玩家到拾枪的 tick 数 (20 tick = 1 s); <code>[ai] enabled = true</code> 时会被 <code>[ai.<tier>] reactionMin/MaxTicks</code> 取代	0..200
<code>combat.giveUpTicks</code>	<code>160</code>	目标不可达时最多坚持多久	20..2400
<code>combat.retreathHealthFraction</code>	<code>0.35</code>	低于此血量比例后就撤而不是死守	0.0..1.0; 被各档 <code>retreatHealthScale</code> 缩放, 或被 <code>[ai.<tier>] retreatHealthFraction</code> 绝对值覆盖
<code>combat.repositionTicks</code>	<code>40</code>	移动到新射击位再交战的时间	10..400
<code>combat.targetMemoryTicks</code>	<code>100</code>	失去视线后记住目标位置多久	0..1200
<code>combat.fireStallTicks</code>	<code>100</code>	FIRE 状态多久没有一枪被接受就放弃该位置	20..2400
<code>combat.logGunAi</code>	<code>true</code>	记录每次状态切换、每个 TaCZ <code>ShootResult</code> 、每个掩体决定	这是枪械集成与掩体系统的证据链开关

5.4 [ai] (门行为 + 总开关)

键名	默认值	作用	备注/推荐范围
<code>ai.closeDoorsBehind</code>	<code>true</code>	单位开出木门并走远后, 把门关回去一次	关掉即「门行为全关」的一半
<code>ai.doorCloseDelayTicks</code>	<code>40</code>	开门后最多保持开着的 tick 数 (20 = 1 s)	0..1200
<code>ai.doorCloseAllRadius</code>	<code>2.0</code>	有任何活体在此半径内就不关门的安全半径 (格)	0.5..16.0
<code>ai.enabled</code>	<code>true</code>	四档智力 profile 的总开关。false = 完全回到「只有全局键」的旧行为	与 <code>[ai.coordination] enabled</code> 、各档 <code>coordination</code> 是三层开关

5.5 [ai.coordination]（小队协同；仅 TROOP / ELITE 参与）

键名	默认值	作用	备注/推荐范围
ai.coordination.enabled	true	协同层总开关。false = 不做小队查询、不分配角色、不做掩体占用	关掉后每只怪完全按自己的档位打
ai.coordination.focusFire	true	集火：小队指定一个目标（票多者胜，平票取最小实体 id），刚获得目标的成员在确认可见后采用它	阵营情报本身不能开战（需 hasLineOfSight）
ai.coordination.overwatch	true	交替掩护：同一时刻恰有一名压制手（window % count == index），按窗口轮换	3 人队形为 0,1,2,0,1,2
ai.coordination.flanking	true	包抄：一部分队员只在「怪—目标」轴的同一侧取掩体	人数夹到 [1, size-1]，总有人守正面
ai.coordination.coverClaims	true	掩体独占：按方块坐标登记占用，别人不再选同一格	到期自动释放
ai.coordination.squadRadius	24.0	找队友的半径（格）	4.0..64.0；约一条街
ai.coordination.squadCacheTicks	40	队员表复用多久后重新查询	5..400；整个协同层的成本控制点
ai.coordination.overwatchWindowTicks	60	一个掩护角色持续多久后轮换	10..600
ai.coordination.overwatchTimeoutTicks	120	压制角色的防死锁上限：超过就退回自己的行为	20..1200
ai.coordination.flankFraction	0.5	参与包抄的队员比例（四舍五入并夹取）	0.0..1.0
ai.coordination.flankOffset	6.0	包抄掩体相对「怪—目标」轴希望偏出多少格	1.0..24.0
ai.coordination.coverClaimTicks	100	掩体占用保留多久	10..1200

5.6 [tactics]（掩体 / 压制 / 推进 / 脱离接触）

键名	默认值	作用	备注/推荐范围
tactics.coverSearchRadius	14	找掩体的半径（格）	3..48
tactics.coverCacheTicks	20	掩体搜索结果复用多久	1..400
tactics.coverSamples	28	每次掩体搜索测试多少候选点	6..128；越高越准越贵
tactics.suppressChance	0.6	目标刚失去视线时改为压制（而非移动）的概率	0.0..1.0；被 suppressChanceScale 缩放
tactics.suppressTicks	60	一次压制持续多久	10..600；被 suppressTicksScale 缩放
tactics.suppressAccuracyMultiplier	0.45	压制时的精度乘数（盲射故意不准）	0.05..1.0
tactics.suppressBurstMultiplier	2.0	压制时点射长度乘数（长而不准）	1.0..6.0
tactics.advanceCoverStep	3.0	推进时新掩体至少要离目标近多少格才值得移动	1.0..16.0；被 advanceCoverScale 缩放
tactics.coverSeekSpeedModifier	1.0	三种「去掩体」移动（换弹躲掩体、换射击位、撤退冲刺）的速度乘数	0.5..3.0
tactics.retreatSprint	false	撤退时是否疾跑。不被任何档位缩放	true 会恢复「疯狂逃跑」
tactics.escapeWithoutCoverSpeedModifier	1.0	找不到掩体、只能裸跑时的速度乘数	0.5..3.0
tactics.moveProgressSampleTicks	20	无进展看门狗的采样间隔	5..200
tactics.moveProgressMinBlocks	0.5	每个采样窗口至少要挪多少格才算有进展	0.05..8.0
tactics.moveProgressRetries	3	卡住的怪在放弃该位置前可重寻路几次	1..20
tactics.underFireTicks	100	被击中后「处于火力下」持续多久（期间偏好更远的掩体）	0..1200
tactics.hurtRetreatChance	0.5	单次中弹立即触发撤退的概率（叠加在血量线之上）	0.0..1.0；可被 [ai.<tier>] hurtRetreatChance 覆盖

键名	默认值	作用	备注/推荐范围
tactics.accuracyNear	1.0	近距离（射程 35% 以内）精度乘数	0.05..2.0
tactics.accuracyMid	0.85	中距离精度乘数	0.05..2.0
tactics.accuracyFar	0.7	远距离精度乘数	0.05..2.0

5.7 [tiers.<pistol|shotgun|rifle|sniper>]

见第 4.3 节（7 个键 × 4 档）。键路径写作 `tiers.pistol.spawnWeight`、`tiers.shotgun.health ... tiers.sniper.burstCooldownTicks`。

5.8 [voice]

键名	默认值	作用	备注/推荐范围
voice.enabled	true	所有语音行的总开关	
voice.idle	true	无目标时自言自语（self-talk 池）	
voice.idleIntervalTicks	400	空闲语音间隔（tick，20 = 1 s；400 = 20 s）	40..24000；取代原版约 6 s 的环境音节奏
voice.idleJitterTicks	80	每次空闲间隔额外随机 tick，避免一群怪齐声	0..2400
voice.contact	true	首次接敌时从 chatter 池说一句	
voice.contactCooldownTicks	200	同一只怪两次接敌语音的最小间隔；也是「跟丢了」行的复用冷却	0..12000
voice.chatter	true	交火中（FIRE/SUPPRESS）按随机间隔继续说话	
voice.chatterMinIntervalTicks	300	该随机间隔下限（300 = 15 s）	20..24000
voice.chatterMaxIntervalTicks	600	该随机间隔上限（600 = 30 s）	40..24000
voice.grenade	true	附近有爆炸物落地时喊话（含原版点燃 TNT 与名字含 grenade 的实体）	
voice.grenadeRadius	12.0	爆炸物要多近才触发	2.0..48.0
voice.grenadeCooldownTicks	200	两次「爆炸物落地」反应的最小间隔	0..12000
voice.grenadeShoutCooldownTicks	40	自己投雷时喊话的独立冷却（与上一键分开）	0..12000；0 = 关闭该冷却
voice.mark	true	丢失目标时说「你跑哪去了」	
voice.death	true	用四条死亡 clip 之一替换原版死亡音（开启时原版死亡音不播）	
voice.volume	1.0	所有语音行的音量（1.0 = 原版怪物音量）	0.0..4.0
voice.familyVolume	["shared=1.0", "usec=1.0", "bear=1.0", "elite=1.0"]	在 voice.volume 之上按家族再乘一次，条目格式 家族=倍率；家族只能取 shared/usec/bear/elite	倍率夹到 0.4；写到未知家族会警告一次并忽略
voice.effectVolume	1.0	非语音效果音（手雷落地/弹跳）的音量乘数	0.0..4.0
voice.pitchMin	0.9	音高带下限。每只怪在出生时抽一次自己的音高并存入 NBT，跨存档保持	硬边界 0.1..2.0；与 pitchMax 一起校验，写反则双双回退出厂值
voice.pitchMax	1.1	音高带上限	硬边界 0.1..2.0
voice.pitchJitter	0.03	每行语音在自身音高上的抖动（0.03 ≈ 3%）	0.0..0.5

5.9 [faction]

键名	默认值	作用	备注/推荐范围
<code>faction.enabled</code>	<code>true</code>	阵营层总开关：同盟、友伤记账、叛变标记、情报网。 <code>false</code> = 三方互相敌对、什么都不共享	
<code>faction.villagersAttackMonsters</code>	<code>true</code>	村民家族（gunner/sniper/usec/elite villager）是否见怪就打（僵尸、骷髅、蜘蛛、苦力怕、原版袭击者等）	谁在名单里是数据包标签： <code>data/tarkovscav/tags/entity_types/faction_village_hostile.json</code>
<code>faction.friendlyFireHitsToAnger</code>	<code>3</code>	同一攻击者在窗口内打中队友几次后，受害者翻脸	1..100
<code>faction.friendlyFireWindowTicks</code>	<code>200</code>	上述计数的滑动窗口（200 = 10 s），超时重新计数	1..72000
<code>faction.betrayalThreshold</code>	<code>3</code>	同一攻击者在窗口内打中队友几次后，攻击者被标记为叛徒	1..100；应 ≥ 上一条
<code>faction.renegadeBroadcastRadius</code>	<code>48.0</code>	「某人是叛徒」这条消息传播多远（格）	4.0..256.0
<code>faction.renegadeDecayTicks</code>	<code>0</code>	叛徒标记多久消退； <code>0</code> = 永久（默认，更有戏）	0..240000
<code>faction.renegadeGlow</code>	<code>true</code>	给叛徒加发光描边	

5.10 [alert]（阵营情报共享：只报方向与距离带，绝不报坐标）

键名	默认值	作用	备注/推荐范围
<code>alert.enabled</code>	<code>true</code>	情报共享总开关；关掉后怪仍正常战斗，只是不通报	
<code>alert.radius</code>	<code>40.0</code>	一条接触报告传播多远（格），也是「远」档边界	4.0..128.0
<code>alert.nearDistance</code>	<code>8.0</code>	NEAR 档上界（格）	1.0..128.0
<code>alert.midDistance</code>	<code>24.0</code>	MID 档上界（格），应 ≥ <code>nearDistance</code>	1.0..256.0
<code>alert.memoryTicks</code>	<code>300</code>	一条报告保持多久（300 = 15 s）	1..24000
<code>alert.maxRecipients</code>	<code>6</code>	每次广播最多通知多少名武装友军（按实体 id 选择）	1..64
<code>alert.broadcastCooldownTicks</code>	<code>40</code>	同一只怪两次广播的最小间隔	1..24000
<code>alert.bearingNoiseDegrees</code>	<code>22</code>	在 45° 扇区量化之上再加的方位误差（度）	0..45
<code>alert.convergeRadius</code>	<code>20.0</code>	报告区域在多近以内，接收者才会走过去	1.0..128.0
<code>alert.convergeMaxAllies</code>	<code>3</code>	针对同一报告最多几人前往，其余原地待命	1..32
<code>alert.surroundStandoff</code>	<code>6.0</code>	前往者站在接触点周围这个半径的环上（均匀分布），不堆在一格	1.0..32.0
<code>alert.minRepathIntervalTicks</code>	<code>60</code>	前往者重新寻路的最小间隔，避免寻路风暴	1..24000

5.11 [rack]（武器放置台）

键名	默认值	作用	备注/推荐范围
<code>rack.enabled</code>	<code>true</code>	方块总开关： <code>false</code> = 既不能取用，也关掉掉落吸收扫描（变成普通单格架子）	方块本身永远可以手工存取
<code>rack.acceptsAnyItem</code>	<code>false</code>	<code>false</code> = 只接受 TaCZ 枪（IGun）、弓、弩，或 <code>#minecraft:swords/#minecraft:axes</code> 内的物品； <code>true</code> = 任何物品（无战斗路径的物品会被 WARN 一次并留在台上，绝不白白消耗）	
<code>rack.takeRadius</code>	<code>8.0</code>	未武装村民/掠夺者要多近才会被武装	1.0..32.0
<code>rack.takeCooldownTicks</code>	<code>200</code>	取走一次后，台子等多久才武装下一个	1..24000
<code>rack.takeCheckIntervalTicks</code>	<code>20</code>	台子每隔多少 tick 找一次取用者	1..1200
<code>rack.priority</code>	<code>"nearest"</code>	多个未武装生物在范围内时先服务谁： <code>nearest</code> （默认）、 <code>villager</code> 或 <code>pillager</code>	偏好而非过滤：首选类型没人时另一种仍会被服务

键名	默认值	作用	备注/推荐范围
<code>rack.absorbDroppedItems</code>	<code>true</code>	是否吸起丢在台子上的物品（玩家手持枪时无法与台子交互，扔过去是唯一的交接方式）	
<code>rack.absorbRadius</code>	<code>0.75</code>	吸收扫描的水平半径（格）	0.0..4.0; 0.75 覆盖方块本身及一点邻格
<code>rack.absorbHeight</code>	<code>1.25</code>	吸收扫描的高度（格）	0.0..4.0
<code>rack.absorbCheckIntervalTicks</code>	<code>8</code>	扫描掉落物的间隔	1..1200
<code>rack.creativeRackEnabled</code>	<code>true</code>	军械台（创造）是否生效； <code>false</code> 时它是惰性的（模板不消耗）	

5.12 [accuracy] (精度档 / warm-up)

键名	默认值	作用	备注/推荐范围
<code>accuracy.enabled</code>	<code>true</code>	总开关。 <code>false</code> = 完全恢复旧行为（从第一发起就用该档精度、无上限）	
<code>accuracy.hardCeiling</code>	<code>0.95</code>	所有档位的绝对上限；1.0 意味着「从不失手」，本模组不允许	0.0..0.95; 手改 profile 上限到 1.0/3.0 也会被夹回
<code>accuracy.profileRookieCap</code>	<code>0.75</code>	rookie 档命中率上限	0.0..1.0
<code>accuracy.profileVeteranCap</code>	<code>0.85</code>	veteran 档上限	0.0..1.0
<code>accuracy.profileEliteCap</code>	<code>0.90</code>	elite 档上限	0.0..1.0
<code>accuracy.profileScav</code>	<code>"rookie"</code>	普通 scav 用哪个档： <code>rookie/veteran/elite</code>	
<code>accuracy.profileGunnerPillager</code>	<code>"rookie"</code>	持枪掠夺者用哪个档	
<code>accuracy.profileGunnerVillager</code>	<code>"rookie"</code>	持枪村民用哪个档	
<code>accuracy.profileSniperTier</code>	<code>"veteran"</code>	档位为 sniper 的单位会被提升到此档（若比类型自带档更好）	
<code>accuracy.warmupShots</code>	<code>8</code>	交火前多少发处于 warm-up（精度乘以下一行的系数）	0..1000
<code>accuracy.warmupMultiplier</code>	<code>0.45</code>	warm-up 期间的精度乘数（ $0.45 \times \text{狙击 } 0.85 = 0.38$ ）	0.0..1.0
<code>accuracy.resetTicks</code>	<code>600</code>	多久没开火后 warm-up 重新开始； <code>0</code> = 终生只算一次	0..240000

5.13 [mods] (随机配件池「改枪王」)

键名	默认值	作用	备注/推荐范围
<code>mods.enabled</code>	<code>true</code>	总开关。 <code>false</code> = 每只怪都是裸枪	
<code>mods.perSlotChance</code>	<code>0.35</code>	每个配件槽被填的概率	0.0..1.0; 0.35 ≈ 通常装一个
<code>mods.fullModChance</code>	<code>0.02</code>	「改枪王」概率：该枪允许的每个槽都填满	0.0..1.0; 0.02 ≈ 五十只里一只
<code>mods.maxPerGun</code>	<code>5</code>	每把枪配件数硬上限（无论槽有多少）	1..8
<code>mods.allowExtendedMag</code>	<code>true</code>	是否允许 <code>EXTENDED_MAG</code> 槽（唯一会改弹匣容量的配件；容量装完后从物品重读，绝不假设）	关掉则怪保持原厂弹匣

5.14 [sniper]

键名	默认值	作用	备注/推荐范围
<code>sniper.enabled</code>	<code>true</code>	总开关： <code>false</code> = 不刷新且关闭蹲点逻辑	

键名	默认值	作用	备注/推荐范围
<code>sniper.followRange</code>	64.0	FOLLOW_RANGE （格）：多远能注意到目标（步枪兵为 35）	16.0..128.0
<code>sniper.discoveredRange</code>	24.0	换位触发 2：目标有视线且近于此距离	4.0..64.0
<code>sniper.shotsBeforeMove</code>	2	打几发后换位	1..20
<code>sniper.minPostDistance</code>	16.0	新射击位离自己（大致也是离目标）至少多远	8.0..64.0
<code>sniper.closeRange</code>	8.0	换位触发 4 兼近战规则：在此距离内后退而不是对拼	2.0..24.0
<code>sniper.moveSpeed</code>	1.1	换位移动速度乘数	0.5..2.0
<code>sniper.relocateTimeoutTicks</code>	200	走去新射击位超过此时限就停下、就地隐蔽并重新锁定目标	20..2400
<code>sniper.minSpawnDistanceFromPlayer</code>	32.0	自然刷新时离任何玩家至少这么远	8.0..128.0
<code>sniper.preferHighGround</code>	true	要求高台刷新（该列比周围地面高出 minElevation 格）	
<code>sniper.minElevation</code>	4	高出周围多少格才算高台	1..32
<code>sniper.spawnWeight</code>	1	狙击掠夺者的自然刷新权重	0..1000; 0 = 禁用自然刷新而禁用该怪
<code>sniper.villagerWeight</code>	1	狙击村民的自然刷新权重（不高于掠夺者 1）	0..1000

5.15 `[client]`（模型 / 挂枪 / 姿势 / 歪头）

键名	默认值	作用	备注/推荐范围
<code>client.useGeckoModel</code>	false	持枪掠夺者是否改用 GeckoLib Bedrock rig（而非原版 villager 模型）	
<code>client.renderScale</code>	0.77	Bedrock rig 系（scav，以及开启 Gecko 时的持枪掠夺者）的渲染缩放；基线 0.7，0.77 = rig 的 1.1×	0.3..2.0
<code>client.villagerRenderScale</code>	1.0	村民系（gunner/sniper/usec/elite villager，共用一个渲染器）的渲染缩放；1.0 = 原版村民大小	0.3..2.0
<code>client.hiddenBones</code>	[]	要隐藏的骨骼列表（出厂为空：2026 重导出已删除占位道具；被保护的骨骼会被忽略并 WARN）	用 <code>/armedmobs client hide <bone></code> 一分法排查
<code>client.logHiddenBones</code>	true	记录每根隐藏骨骼及其原因（每只怪一次）	
<code>client.gunAnchorBone</code>	"RightHandLocator"	主手枪挂载的骨骼名	回退链：本名 → RightHandLocator → RightHand
<code>client.gunMountRifleRotation</code>	[0, 0, 0]	步枪类挂枪的额外旋转（度， [x, y, z] ，按 X→Y→Z 应用）	默认中性：显示上下文已修正姿态
<code>client.gunMountRifleOffset</code>	[0, 0, -0.7]	步枪类挂枪的偏移（方块，锚点帧 [x, y, z] ）；-0.7 是用户实测的向前滑移	
<code>client.gunMountRifleScale</code>	1.0	步枪类额外统一缩放（TaCZ 自身已乘 0.6）	0.05..4.0
<code>client.gunMountPistolRotation</code>	[0, 0, 0]	手枪类挂枪旋转（规则同步枪）	
<code>client.gunMountPistolOffset</code>	[0, 0, -0.7]	手枪类挂枪偏移（与步枪同一 normalisedHand 帧，故同值）	觉得手枪偏高可自己加回 -0.125 ： [0, -0.125, -0.7]
<code>client.gunMountPistolScale</code>	1.0	手枪类额外统一缩放	0.05..4.0
<code>client.gunMountDisplayContext</code>	"THIRD_PERSON_RIGHT_HAND"	持枪渲染用的 ItemDisplayContext （TaCZ 渲染器按此分支）	接受任意 ItemDisplayContext 名，大写。 FIXED 会镜像翻转并放大（物品展示框布局）， FIRST_PERSON_* 与 THIRD_PERSON_LEFT_HAND 什么都不画
<code>client.gunAnchorMode</code>	"normalisedHand"	锚点帧模式： normalisedHand （转成原版手持帧）或 locatorAnimated	未识别的值会 WARN 并回到 normalisedHand

键名	默认值	作用	备注/推荐范围
client.gunOffhandAnchorBone	"LeftHandLocator"	副手物品挂载的骨骼名	回退链: 本名 → LeftHandLocator → LeftHand
client.renderOffhandItem	true	是否像原版 ItemInHandLayer 那样画出副手物品 (副手为空则什么都不画)	
client.gunTwoHandedSupport	false	实验性: 把主手枪也画在副手锚点上, 便于对齐双手握持 (会把枪画两次, 未重合时 z-fighting)	多数 rig 的 tac:hold:*/tac:aim:* clip 已把手摆好, 不需要
client.gunMountOffhandRotation	[0, 0, 0]	副手锚点上所画物品的额外旋转 (度, 按 X→Y→Z)	也用于双手支撑副本
client.gunMountOffhandOffset	[0, 0, 0]	副手锚点额外偏移 (方块)	
client.gunMountOffhandScale	1.0	副手锚点额外统一缩放	0.05..4.0
client.logGunMount	true	每只怪记录一次「枪挂在哪根骨、用什么变换」	
client.gunnerVillagerArmPitch	-57.0	武装村民瞄准 (RAISED) 时手臂在原版静止姿势之上的俯仰偏移 (度): 0 = 原版抱臂, 负值抬高	-180..180
client.gunnerVillagerHoldArmPitch	0.0	持枪但未瞄准 (LOWERED/IDLE) 时的同一偏移; 0 = 原版抱臂	-180..180
client.gunnerVillagerReloadArmPitch	0.0	换弹姿势的手臂偏移	-180..180
client.gunnerVillagerHunkerArmPitch	0.0	撤退 (HUNKERED) 姿势的手臂偏移; 正值把手臂压向身体	-180..180
client.gunnerVillagerGunOffset	[0, 0.06, -0.09]	枪相对村民手臂块的偏移 (方块, [x, y, z]; -Z 向前, +Y 向上, +X 为村民右侧)	
client.gunnerVillagerGunRotation	[5, 0, 0]	枪在手臂帧内的旋转 (度, [pitch, yaw, roll], 按 X→Y→Z); 出厂 5,0,0 (源码常量; README §5j 历史提到 -40)	枪的倾角 = 手臂俯仰 + 本键 X + 该姿势增量 - 90
client.gunnerVillagerIdleGunRotation	[-2, 0, 0]	仅闲置时叠加到上键的旋转增量	
client.gunnerVillagerReloadGunRotation	[0, 0, 0]	仅换弹时叠加的旋转增量	
client.gunnerVillagerHunkerGunRotation	[0, 0, 0]	仅撤退时叠加的旋转增量	
client.gunnerVillagerIdleGunOffset	[0, 0, 0]	仅闲置时叠加到 gunnerVillagerGunOffset 的位置增量	
client.gunnerVillagerReloadGunOffset	[0, 0, 0]	仅换弹时的位置增量	
client.gunnerVillagerHunkerGunOffset	[0, 0, 0]	仅撤退时的位置增量	
client.gunnerVillagerGunScale	1.0	村民所持枪的额外统一缩放 (乘在 TaCZ 自身 0.6 之上)	0.05..4.0; 最终大小 ≈ renderScale × 本键 × 0.6
client.gunnerVillagerGunAnchor	"arms"	枪挂在哪个模型部件: arms (跟随抱臂动画, 默认) 或 body (更稳但不跟手臂动画)	
client.hideGunWhenIdle	false	武装村民站立不动 (LOWERED) 时是否隐藏手上的枪	
client.modelRenderType	"cutout"	模型渲染类型	
client.cullingBoxPadding	1.0	剔除盒外扩 (方块)	0.0..8.0
client.modelLayering	"upperLower"	模型分层: upperLower (腿/上半身两层控制器, 源码默认) 或 single (单控制器)	两种模式都是每帧一次几何提交
client.logRenderStats	false	记录每帧渲染统计	

键名	默认值	作用	备注/推荐范围
<code>client.logGlState</code>	<code>false</code>	记录 GL 状态（贴图状态泄露排查用）。 [gldebug] 行还带 guard 恢复的四项： <code>backFunc</code> （背面模板 func）、 <code>clear</code> （stencil clear 值）、 <code>activeUnit</code> （活动纹理单元）、 <code>shaderTex0</code> （RenderSystem 0 号 sampler），见 README 5k	
<code>client.poseSource</code>	<code>"auto"</code>	谁拥有瞄准姿势骨骼： <code>auto</code> （逐骨：作者关键帧跟随视角就归 clip，否则归代码）、 <code>code</code> （代码写 <code>UpperBody/Head</code> ，加 <code>molangVariables=off</code> 即旧行为）、 <code>clips</code> （全归 clip）	未识别值 WARN 并回退 <code>auto</code>
<code>client.molangVariables</code>	<code>"pitch"</code>	喂给 rig 自身 <code>ysm.*query.*</code> 瞄准变量的范围： <code>off/pitch/all</code> （ <code>all</code> 含 yaw）	未识别值回退 <code>pitch</code>
<code>client.torsoYawShare</code>	<code>0.25</code>	代码把多少看向偏航放在胸腔上（其余给头，瞄准落点不变）	0.0..1.0
<code>client.logPoseWriters</code>	<code>false</code>	每帧输出一行 [pose]，写明每根姿势骨骼的写入者	
<code>client.leanEnabled</code>	<code>true</code>	玩家歪头总开关。 <code>false</code> = 歪头键完全无效（不改相机、不屏蔽按键、不偏移弹道起点）	
<code>client.leanMaxOffset</code>	<code>0.6</code>	满歪时相机侧移距离（方块），也是自己弹道的枪口侧移距离	0.0..1.5
<code>client.leanRollDegrees</code>	<code>12.0</code>	满歪时视角滚转（度）；0 = 只平移	0.0..45.0
<code>client.leanInvertOffset</code>	<code>false</code>	只反转侧移方向	
<code>client.leanInvertRoll</code>	<code>false</code>	只反转视角滚转（移动看起来对、地平线交错方向时用）	
<code>client.leanSpeedTicks</code>	<code>5</code>	歪头过渡所需 tick	1..20
<code>client.leanSuppressVanillaKeys</code>	<code>true</code>	歪头键占用 Q/E 时屏蔽这两个原版键（含短按），按「绑定的键」而非全局屏蔽	<code>false</code> = 完全不动原版键（歪头同时会丢物品/开背包）
<code>client.tapThresholdTicks</code>	<code>5</code>	短于多少 tick 视为「点按」	1..40
<code>client.startMode</code>	<code>"immediate"</code>	歪头起始模式	
<code>client.replayVanillaOnTap</code>	<code>true</code>	点按（短于 <code>tapThresholdTicks</code> ）时是否在松手时由本模组补做原版动作（E 开背包、Q 丢一个）	<code>false</code> = 点按什么都不做

5.16 [client.headAccessories]

键名	默认值	作用	备注/推荐范围
<code>client.headAccessories.hatAccessory</code>	<code>"keepOne"</code>	帽子处理： <code>keepOne</code> （保留 <code>keptHatBone</code> ，隐藏其余帽子）、 <code>hideAll</code> 、 <code>showAll</code>	未识别值 WARN 并按 <code>keepOne</code>
<code>client.headAccessories.hatBones</code>	<code>["Hat1", "Hat2", "hat3"]</code>	哪些骨骼算帽子（只对 <code>keepOne/hideAll</code> 有意义）	当前 rig 只有 <code>Hat2</code> ，其余名字报一次「不在本 rig」后忽略
<code>client.headAccessories.keptHatBone</code>	<code>"Hat2"</code>	<code>keepOne</code> 保留的那顶帽子	不在 <code>hatBones</code> 里时会 WARN 且所有帽子都被隐藏
<code>client.headAccessories.eyegearAccessory</code>	<code>"none"</code>	戴哪件眼部装备； <code>none/off/false/空</code> = 全部隐藏	
<code>client.headAccessories.eyegearBones</code>	<code>["Glass", "YanJing"]</code>	哪些骨骼算眼部装备	
<code>client.headAccessories.cigarette</code>	<code>false</code>	是否画出嘴里的香烟	
<code>client.headAccessories.cigaretteBone</code>	<code>"Yan"</code>	承载香烟的骨骼名（2026 重导出已删除 <code>Yan</code> ，开关默认关闭）	

键名	默认值	作用	备注/推荐范围
<code>client.headAccessories.extraHiddenBones</code>	<code>[]</code>	额外隐藏的骨骼（诊断二分用，默认什么都不藏）	等价于 <code>/armedmobs client hide <bone></code>
<code>client.headAccessories.headRestPitchDegrees</code>	<code>20.0</code>	头部静止俯仰（度）	-90..90
<code>client.headAccessories.headRestPitchStates</code>	<code>["idle"]</code>	头部静止俯仰在哪些 AI 状态生效； <code>any/*/all</code> = 全部	合法状态： <code>idle</code> 、 <code>alert</code> 、 <code>advance</code> 、 <code>aim</code> 、 <code>fire</code> 、 <code>suppress</code> 、 <code>reload</code> 、 <code>bolt</code> 、 <code>reposition</code> 、 <code>retreat</code>

5.17 [killFeed]

键名	默认值	作用	备注/推荐范围
<code>killFeed.enabled</code>	<code>true</code>	HUD 元素总开关。false = 一行都不画（服务端也不再产生记录）	
<code>killFeed.mode</code>	<code>"involved"</code>	谁能看到： <code>involved</code> （只有涉事玩家 / 附近的玩家，见 <code>radius</code> ）等	取值集合见 README §5u（未核对：源码默认串为 <code>involved</code> ，其余模式名未在本次阅读范围内逐个确认）
<code>killFeed.radius</code>	<code>64.0</code>	玩家要多近才看到「别人之间」的击杀行（仅 <code>mode = involved</code> 使用）	8.0..512.0
<code>killFeed.showMobKills</code>	<code>true</code>	是否播报双方都不是玩家的击杀（含本模组单位互杀）	
<code>killFeed.showPlayerKills</code>	<code>true</code>	是否播报有玩家参与的击杀	
<code>killFeed.showEnvironmentDeaths</code>	<code>true</code>	是否播报环境死亡	
<code>killFeed.lineDurationTicks</code>	<code>100</code>	一行停留多久	20..1200
<code>killFeed.maxLines</code>	<code>5</code>	最多同时几行	1..20
<code>killFeed.position</code>	<code>"top_center"</code>	HUD 位置	
<code>killFeed.scale</code>	<code>1.0</code>	文字缩放；长名字被截断而非换行	0.5..2.0
<code>killFeed.maxPerSecond</code>	<code>4</code>	节流：每秒最多给同一玩家几行	1..40
<code>killFeed.dedupTicks</code>	<code>40</code>	同一「凶手 + 受害者 + 武器」在这段时间内重复则丢弃	0..400

5.18 [troops]（USEC 村民 / BEAR 掠夺者 / 优质村民 / 优质掠夺者）

键名	默认值	作用	备注/推荐范围
<code>troops.armorEnabled</code>	<code>true</code>	掷骰护甲等级的总开关（false 时仍掷但不用）	
<code>troops.armorReductionPerClass</code>	<code>0.10</code>	每级甲减伤：0.10 = 1 级减 10%，6 级减 60%	0.0..0.25；这些怪故意不穿原版甲，否则会二次减伤
<code>troops.minArmorClass</code>	<code>1</code>	新刷新部队可掷到的最低甲等	1..6
<code>troops.maxArmorClass</code>	<code>6</code>	最高甲等	1..6
<code>troops.usecVillagerWeight</code>	<code>1</code>	USEC 村民自然刷新权重（与 <code>add_scavs.json</code> 相等）	0..1000
<code>troops.bearPillagerWeight</code>	<code>1</code>	BEAR 掠夺者自然刷新权重（与 <code>add_scavs.json</code> 相等）	0..1000
<code>troops.eliteVillagerWeight</code>	<code>1</code>	优质村民自然刷新权重（故意最稀有）	0..1000
<code>troops.elitePillagerWeight</code>	<code>1</code>	优质掠夺者自然刷新权重（故意最稀有）	0..1000

5.19 [ricochet]（硬目标：铁傀儡等）

键名	默认值	作用	备注/推荐范围
<code>ricochet.enabled</code>	<code>true</code>	总开关。 <code>false</code> = 硬目标规则完全失效（子弹伤害不减免、不跳弹）	
<code>ricochet.gunDamageMultiplier</code>	<code>0.2</code>	未跳弹时枪械伤害乘数（0.2 = 减 80%），在 TaCZ 交出的那一击上应用、先于原版护甲	0.0..1.0
<code>ricochet.chance</code>	<code>0.3</code>	命中硬目标时跳弹的概率：跳弹那一下零伤害、弹体反射、播放金属声与火花	0.0..1.0; 0 = 从不跳，1 = 总是
<code>ricochet.maxBouncesPerBullet</code>	<code>1</code>	同一弹体最多跳几次后按普通处理	0..8; 0 = 仍减伤但永不反射
<code>ricochet.includeArrows</code>	<code>false</code>	原版箭是否也算「枪械伤害」（它们不在 TaCZ 的 <code>#tacz:bullets</code> 伤害标签里）	
<code>ricochet.soundVolume</code>	<code>0.8</code>	跳弹音（原版 <code>TRIDENT_RICOCHET</code> ）音量	0.0..4.0
<code>ricochet.sparks</code>	<code>true</code>	跳弹是否产生火花粒子（ <code>CRIT + ELECTRIC_SPARK</code> ）	
<code>ricochet.log</code>	<code>false</code>	是否记录每次跳弹判定	

5.20 [grenades]（投掷物总段）

键名	默认值	作用	备注/推荐范围
<code>grenades.enabled</code>	<code>true</code>	总开关。 <code>false</code> = 物品无法投掷（右键无反应）、怪也不扔，已点燃的引信会被拆除而不爆炸	
<code>grenades.terrainDamage</code>	<code>false</code>	<code>false</code> = 地形永不被破坏（ <code>ExplosionInteraction.NONE</code> 的原版爆炸：伤实体、不动方块）	
<code>grenades.throwChargeTicks</code>	<code>20</code>	右键蓄力多久算满力投掷；更短是抛投，且蓄力期间引信在烧（即「烹饪」）	4..100
<code>grenades.minThrowSpeed</code>	<code>0.6</code>	无蓄力时的投掷速度（格/tick）	0.2..3.0
<code>grenades.maxThrowSpeed</code>	<code>1.6</code>	满蓄力投掷速度	0.4..4.0
<code>grenades.cookWhileHolding</code>	<code>true</code>	雷在手里时引信是否继续烧	
<code>grenades.friendlyFire</code>	<code>false</code>	<code>false</code> = 手雷永不伤投掷者与同阵营（ <code>Faction.allies</code> ），小队不会自灭； <code>true</code> = 伤所有人。怪在落点有队友时本就会拒绝投掷	爆炸仍会推动不受伤的实体（击退不过滤）
<code>grenades.playerSelfDamage</code>	<code>true</code>	玩家是否会被自己的手雷伤害	
<code>grenades.blastDamagePerPower</code>	<code>12.0</code>	爆炸本身在零距离每点 <code>blast power</code> 的伤害（爆炸半径仍是 <code>blastPower × 2</code> 格，本键只加倍伤害不加范围）	0.0..100.0; 想让爆炸更大就调 <code>grenades.he.blastPower</code>
<code>grenades.impactSound</code>	<code>true</code>	投出的雷落地/撞墙时是否播放落地音（音源为 <code>sounds/effect/grenade_land.ogg</code> ，由语音清单注册）	
<code>grenades.impactSoundMaxPerGrenade</code>	<code>3</code>	同一颗雷最多播几次落地音	1..20
<code>grenades.impactSoundCooldownTicks</code>	<code>8</code>	同一颗雷两次落地音的最小间隔（避免滚下坡时连响）	0..100

5.21

[grenades.frag] · [grenades.he] · [grenades.smoke] · [grenades.flash] · [grenades.flashShort]

键名	默认值	作用	备注/推荐范围
grenades.frag.fuseTicks	60	破片手雷引信	4..400
grenades.frag.fragmentCount	24	从爆点射出的破片射线数（每条在第一个方块处停下，所以墙是真掩体）	0..200
grenades.frag.fragmentDamage	14.0	零距离破片伤害（未计衰减与护甲）	0.0..100.0
grenades.frag.fragmentRadius	10.0	破片飞行距离（格），伤害线性衰减到 0	1.0..32.0
grenades.frag.fragmentArmorPierce	0.35	0..1：破片伤害中无视护甲的比例	0.0..1.0
grenades.frag.fragmentStep	0.5	破片射线步长（格），越小越准越贵	0.2..2.0
grenades.he.fuseTicks	80	HE 手雷引信（更重的投掷，出厂更长）	4..400
grenades.he.blastPower	4.0	HE 爆炸强度（半径 = $\text{blastPower} \times 2$ 格）	0.0..20.0
grenades.he.fragmentCount	8	HE 破片数（少于破片手雷，因为靠爆炸）	0..200
grenades.he.fragmentDamage	8.0	HE 零距离破片伤害	0.0..100.0
grenades.smoke.fuseTicks	40	烟雾弹引信	4..400
grenades.smoke.radius	4.0	烟雾半径（格）	1.0..16.0
grenades.smoke.durationTicks	300	烟雾持续	20..2400
grenades.flash.fuseTicks	40	闪光弹引信	4..400
grenades.flash.flashRadius	12.0	闪光作用半径（格），超出什么都不发生	1.0..48.0
grenades.flash.flashIntensity	1.0	零距离白色遮罩亮度	0.0..1.0
grenades.flash.playerBlindTicks	100	零距离玩家致盲时长	10..1200
grenades.flash.mobBlindTicks	120	零距离生物致盲时长（原版失明，枪械 AI 会转成恐慌开火）	10..2400
grenades.flash.lookAwayFactor	0.5	背对爆点者的时长与强度乘数	0.0..1.0
grenades.flash.blindsMobs	true	闪光是否致盲生物	
grenades.flash.panicFire	true	true = 被闪的怪恐慌开火（不需要目标，朝最后已知方向或随机方向，锥面更宽）	
grenades.flash.panicSpreadMultiplier	8.0	恐慌开火的散布乘数	1.0..40.0
grenades.flash.panicBurstTicks	4	恐慌射击的间隔 tick（越小越狂）	1..40
grenades.flashShort.fuseTicks	20	短引信闪光弹引信	2..400
grenades.flashShort.blindFactor	0.75	短引信闪光的致盲倍率	0.05..1.5

5.22 [grenades.mob]（怪投雷）

键名	默认值	作用	备注/推荐范围
grenades.mob.enabled	true	持枪怪是否携带并投掷手雷	
grenades.mob.carryChance	0.35	每只持枪怪出生时带雷的概率	0.0..1.0
grenades.mob.maxPerMob	2	单只怪最多带几颗	1..8
grenades.mob.cooldownTicks	200	同一只怪两次投掷的最小间隔	20..2400
grenades.mob.minRange	6.0	目标近于此距离（格）绝不投（否则等于自爆）	2.0..32.0
grenades.mob.maxRange	20.0	目标远于此距离绝不投（会打不到）	4.0..48.0
grenades.mob.allySafetyRadius	4.5	预测落点周围检查队友的半径（格）	1.0..16.0
grenades.mob.allySafetyMax	0	该半径内最多允许几名队友（超过就拒绝投）；0 = 有一名队友就不投	0..8
grenades.mob.arcSamples	13	弹道求解的采样数	3..64

键名	默认值	作用	备注/推荐范围
<code>grenades.mob.requireClearArc</code>	<code>true</code>	只投「解出的弧线真的落在目标上」的雷； <code>false</code> = 弧被挡也照投（会打到掩体）	
<code>grenades.mob.maxLaunchPitchDegrees</code>	<code>45.0</code>	求解时的最大发射仰角	5.0..80.0
<code>grenades.mob.retryCooldownTicks</code>	<code>40</code>	一次「扣住不投」（弧被挡或落点有队友）后，隔多久再解算	1..600
<code>grenades.mob.resupplyEnabled</code>	<code>true</code>	打完雷的怪是否走向有雷的武器放置台补给	
<code>grenades.mob.resupplyRadius</code>	<code>24.0</code>	愿意走多远去补雷（格），同时是搜索半径（搜索走区块方块实体表，不再按体积收费）	4.0..96.0
<code>grenades.mob.resupplyCooldownTicks</code>	<code>200</code>	取到一颗后隔多久再去取下一颗	20..2400
<code>grenades.mob.resupplySearchCooldownTicks</code>	<code>40</code>	没找到放置台时的搜索冷却（旧行为是每 tick 搜一次，实测每只空闲怪约 1.25 ms 服务端 tick）	1..1200
<code>grenades.mob.rackPriority</code>	<code>"grenade"</code>	补给时优先找哪种放置台	

5.23 `[ai.scav]` / `[ai.sniper]` / `[ai.troop]` / `[ai.elite]`

见第 4.2 节（26 个键 × 4 档）。键路径写作 `ai.scav.reactionMinTicks` ... `ai.elite.retreatHoldTicks`。

5.24 `[command]`（指挥系统：三件阵营道具 + 信号棒 + 信号点方块）

阵营锁定：`village_command_tool` 只影响 `#tarkovscav:faction_village`，`illager_command_tool` 只影响 `#tarkovscav:faction_illager`，`scav_command_tool` 只影响 `#tarkovscav:faction_scav`；三者互不越界（判定只看实体类型标签）。标记中立，任何一件道具都能把任何标记当作目标。

键名	默认值	作用	备注/推荐范围
<code>command.enabled</code>	<code>true</code>	指挥系统总开关	<code>false</code> 时道具变为惰性并给出提示，已有命令也不再推进
<code>command.radius</code>	<code>32.0</code>	指挥道具的影响半径（格）：标记周围这个范围内的同阵营、可被本模组指挥的单位才会接到命令	4.0..256.0
<code>command.speedScale</code>	<code>0.65</code>	推进速度倍率（1.0 = 该单位自己的步行速度）；永远不冲刺	0.05..1.0
<code>command.arrivalRadius</code>	<code>4.0</code>	进入标记这个半径（格）内即视为到达，命令自动解除	0.5..32.0
<code>command.stickDurationTicks</code>	<code>6000</code>	信号棒标记（以及道具右键地面产生的临时标记）的存活时间，单位 tick；6000 = 5 分钟。信号点方块的标记是永久的，不受此键影响	20..240000
<code>command.coordination</code>	<code>true</code>	协同推进：按实体 id 错峰出发，并按现有 <code>SquadCoordinator</code> 的交替掩护轮换跃进（一个成员停、其余前进）。 <code>false</code> = 全体同 tick 起步、各自直接走	
<code>command.maxMarks</code>	<code>12</code>	单个维度同时存在的标记上限。新标记一定进得去，超出时最旧的那个失效	1..64

右键语义（`CommandToolItem`）：右键方块 = 新建标记（取第一个空闲字母）→ 设为当前标记 → 立刻下令；潜行+右键方块 = 只新建标记；右键空气 = 向当前标记下令；潜行+右键空气 = 按 A→B→C 循环切换当前标记并在聊天栏显示「当前标记：B (x, y, z)」。当前标记被删除/过期后自动回落到第一个存活标记；本维度一个标记都没有时，两种右键都会明确提示而不是静默失败。命令写在单位自己的持久化 NBT（`tarkovscav:advanceOrder`）里，重登、区块卸载、服务器重启都不丢；标记消失则命令自动失效。战斗中与撤退中的单位不接受推进（战斗与自保优先），撤退结束后命令若仍在则继续推进。

5.25 [garrison]（一次性城市驻军 / 按楼分配阵营）

每个城市固定刷几队 TROOP 级小队，只在玩家第一次靠近时刷一次，之后永不重刷；阵亡的驻军成员跨重启也不会复活。账本是一个 `SavedData` (`data/tarkovscav_garrison.dat`)，键 = 维度 + 城市标识（结构 id 或区域名 + 包围盒中心按 16 格取整），所以同一个 `city_small` 在主世界与 `tarkovscav:urban_wasteland` 各算一个城市。

阵营的单位是楼，不是城：一座城先掷一个主导阵营（村民 / 掠夺者），再按 `garrison.cityDominantFactionChance` 决定整城统一还是割据；割据时每栋楼独立 50/50 掷一次，因此同一座城（甚至同一个区块里的两栋楼）可以分属不同阵营。楼行在账本里的键是 维度 | 城市标识 # 楼 id，楼 id 形如 `0:north_west`。城市有多个 jigsaw 件时每个件算一栋楼；出厂的单件城市由生成器从同一份 layout 写出 `data/tarkovscav/city_buildings/<结构名>.json`（每栋楼一条矩形），运行时按该 piece 自己的随机旋转映射到世界坐标；两者都没有时整座城算一栋楼。`CityBuildings.indexAt` 先做包含判定，不在任何楼里（街道）则取最近的楼。阵营一旦记录就永不改变，编辑概率只影响尚未决定的城市。

键名	默认值	作用	备注/推荐范围
<code>garrison.enabled</code>	<code>true</code>	总开关； <code>false</code> 只停止新的驻军放置，已放置的保留、账本保留	
<code>garrison.squadsPerCity</code>	<code>-1</code>	每城小队数。 <code>-1</code> = 使用尺寸公式 $1 + \max(\text{宽}, \text{深}) / 48$ ，结果夹在 <code>1..6</code> ；填 <code>1..6</code> 则所有城市都用这个固定值	<code>-1..6</code>
<code>garrison.squadSizeMin</code>	<code>1</code>	小队人数下限（用户要求「一队通常 1-5 人」）	<code>1..16</code>
<code>garrison.squadSizeMax</code>	<code>5</code>	小队人数上限；手改 toml 把 min 写得比 max 大时，读取时自动交换	<code>1..16</code>
<code>garrison.eliteLeaderChance</code>	<code>0.2</code>	每支小队额外多一名该楼阵营的 ELITE 队长（ <code>tarkovscav:elite_villager</code> / <code>elite_pillager</code> ）的概率； <code>0</code> = 不要队长	<code>0.0..1.0</code>
<code>garrison.triggerRadius</code>	<code>64.0</code>	玩家距离城市包围盒多近时触发（格，算的是到盒子的最近点，不是到中心）	<code>16.0..256.0</code>
<code>garrison.checkIntervalTicks</code>	<code>100</code>	触发器多久跑一次（tick）。刻意是粗粒度：只检查已加载区块，不会因为玩家走动而加载地形	<code>20..6000</code>
<code>garrison.friendlyCityChance</code>	<code>0.5</code>	城市主导阵营是村民的概率； <code>0</code> = 全图偏掠夺者， <code>1</code> = 全图偏村民。骰子 = 世界种子 + 城市键，首见即记账，之后改这个值不会翻转已有城市	<code>0.0..1.0</code>
<code>garrison.cityDominantFactionChance</code>	<code>0.5</code>	整座城统一（每栋楼都取主导阵营）的概率；否则城市割据、每栋楼独立 50/50。默认约 56% 的城市统一； <code>1.0</code> 精确恢复「一城一阵营」， <code>0.0</code> 几乎每城混编	<code>0.0..1.0</code>
<code>garrison.rewriteCitySpawners</code>	<code>true</code>	首次确定阵营时，把城内每个 <code>minecraft:spawner</code> 的 <code>SpawnData/SpawnPotentials</code> 改写成它所在那栋楼的 TROOP id（村民楼 <code>usec_villager</code> ，掠夺者楼 <code>bear_pillager</code> ），其它键原样保留；只碰已加载区块、绝不碰盒子外的刷怪笼，整盒覆盖后才记账	
<code>garrison.factionSpawnFilter</code>	<code>true</code>	自然刷怪按楼过滤阵营：村民楼不刷掠夺者阵营单位，掠夺者楼不刷村民阵营单位（用实体类型标签，含原版村民/掠夺者/铁傀儡）。SCAV 是第三方，两种城都放行；没有账本的城市会在刷怪检查时即时掷骰记录，不会误封一切； <code>/summon</code> 与刷怪蛋默认豁免	

组成与放置：每名普通成员的阵营取自它自己站立点所在的那栋楼（村民楼 `tarkovscav:usec_villager`，掠夺者楼 `tarkovscav:bear_pillager`，均为 `AiProfile.Tier.TROOP`），队长取该楼阵营的 ELITE（`elite_villager` / `elite_pillager`）。放置点必须是「脚下实心、自身与头顶两格无碰撞、不在门里」的站立点，优先建筑内部（第一遍只收看不到天空的点），彼此至少隔 2 格。同一支小队共享一个 `tarkovscav:squadId`（写在持久化数据里）并通过 `gun/SquadCoordinator` 各自认领一个掩体方块，所以它们是真正的一支小队，而不是一堆互不相干的

怪；`SquadCoordinator` 的小队成员判定只在自身带 `squadId` 时才要求对方 `id` 相同，因此普通怪物的行为完全不变。每个城市刷出时写一行 `[garrison] <城市> -> N squads / M units [dominant <阵营>: {各阵营计数}]` 日志，账本（含每栋楼的阵营）可用 `/armedmobs garrison` 或 `/armedmobs city faction` 查看与覆写；一个站立点都找不到时不写账本，下一次检查会重试。

后续批次（本次未实现）：阵营兵力池（20-100，按城市大小缩放）、按阵营的 HUD 血条、击杀扣池、池归零后该区域该阵营停止刷武装单位。`GarrisonData` 已留 TODO 说明所需的键形：按城按阵营为 维度|城市标识|阵营名（字段 `strength / max / 最后变化 tick`），按楼则挂在 维度|城市标识#楼 `id` 上。

5.26 [shield]（VANT 防弹盾牌：正面挡枪弹，手雷照打）

`tarkovscav:vant_shield`（「VANT 防弹盾牌」）：拿在主手或副手即**被动**生效（没有右键举盾状态），正面锥形内的枪弹（`TaCZ #tacz:bullets`）减免，手雷与所有爆炸默认完全不挡；耐久 500，扣到 0 当场碎掉消失（音效 + 粒子 + 提示）。规则在 `combat/VantShieldHandler`；「这是不是枪械伤害」复用铁傀儡跳弹的 `HardTarget.isGunfire`（同一个 `#tacz:bullets` 伤害类型标签），近战/摔落/火焰不受影响。来源位置未知时按「在正面」处理（对持有者更安全）；角度恰好等于半角算挡下。合成：`data/tarkovscav/recipes/vant_shield.json`（1 铁块 + 5 铁锭 + 1 玻璃）。

键名	默认值	作用	备注/推荐范围
<code>shield.enabled</code>	<code>true</code>	总开关。false = 完全惰性（不减伤、不扣耐久、不会碎）	
<code>shield.bulletReduction</code>	<code>0.99</code>	正面锥形内的枪弹伤害乘数 = <code>1 - 此值</code> ；0.99 = 只剩 1%	0.0..1.0；1.0 = 全免
<code>shield.frontAngleDegrees</code>	<code>90</code>	正面锥形半角（度），从持有者视线量起；90 = 180 度正面，45 = 90 度，180 = 全向	0.0..180.0
<code>shield.durabilityPerBlockedHit</code>	<code>2.0</code>	每点耐久抵多少被挡下的伤害：扣耐久 = <code>max(1, ceil(被挡伤害 / 此值))</code>	0.1..100.0
<code>shield.protectFromExplosions</code>	<code>false</code>	false（默认）= 爆炸（手雷/TNT/苦力怕） 永不减免 ；只有 true 才会把爆炸也送进锥形判定	

5.27 [ladder]（爬梯：九个武装单位自己上下梯子）

原版里怪物**不会爬梯子**：`WalkNodeEvaluator` 不为梯子生成垂直边，`Mob` 也没有「贴梯上行」的分支（只有玩家 `LocalPlayer#aiStep` 有）。所以生成器造出来的梯井对怪物等同于一面墙。本模组自己补上三层：**垂直链接**（目标在另一层时在半径内找合法梯井，把路线拆成走到梯脚→爬→出开口）、**移动**（贴梯、按速度上下、到层才把控制权交回寻路器）、**意图**（只在目的地确实在另一层且找到梯井时才启动的 goal）。爬梯期间不开火、不换弹（枪口对着墙），战斗与撤退优先于爬梯；被打断时**落在某一层**，绝不挂在半空。核心在 `gun/LadderSearch`（纯逻辑，可无游戏执行）+ `gun/LadderClimbGoal`。诊断：生成器的竖井报告（`LADDER_REPORT`）。

键名	默认值	作用	备注/推荐范围
<code>ladder.enabled</code>	<code>true</code>	总开关。false = 回到「完全不会用梯子」的旧行为（也方便 A/B 对比）	
<code>ladder.climbSpeed</code>	<code>0.15</code>	上行速度（格/tick）。玩家约 0.2，默认比玩家慢，追击爬梯才像样	0.01..1.0
<code>ladder.downSpeed</code>	<code>0.10</code>	下行速度（格/tick）。故意比上行慢：全速下坠看着像掉下去，而且会甩开队友	0.01..1.0
<code>ladder.searchRadius</code>	<code>8</code>	愿意为找梯井绕多远（格）；8 覆盖隔壁房间或转角，再大就会出现「为了爬楼横穿整栋」	2..32
<code>ladder.maxHeight</code>	<code>48</code>	单次连续攀爬的最大高度（格），超过就停下重新规划；防止「通向虚无的梯子」让单位永远往上爬	4..256

键名	默认值	作用	备注/推荐范围
<code>ladder.combatWhileClimbing</code>	<code>false</code>	是否允许在梯子上开火/换弹。默认禁（面朝梯子，枪口在墙里）；手雷无论如何不在梯子上扔	
<code>ladder.fallDamageInShaft</code>	<code>false</code>	梯井内是否恢复摔落伤害。默认关闭（从梯子上滑一两格不算摔）；被打出梯井（脚下已无梯子）恢复原版摔落规则	

5.28 [capture]（主世界城市争夺战：兵力池 + 血条；废土无效）

用户口径：「占领城市这种玩法只在主世界有」「不被占领会一直刷几个阵容的单位，直到一方兵力消失（思路 and 战地一样）」「废土世界完全是各种人乱斗，不需要占领」。所以两个维度规则故意不同：主世界按城建兵力池、扣到 0 该阵营永久停刷、幸存方占领；废土不建池、不判占领、永不出血条（照旧按楼分阵营乱斗）。池与占领标记存在 `data/tarkovscav_garrison.dat` 的 `pools` 列表里（<维度>|<城>|<阵营> + `strength/max/captured`），重启不丢。开池公式：`clamp(poolMin + 建筑数 × poolPerBuilding, poolMin, poolMax)`（同一座城两方开局一样多）。「停止刷新」是运行时拦截（`MobSpawnEvent.PositionCheck` 对自然刷怪与刷怪笼同时生效 + 驻军逐单位问一次），从不改写或熄灭你的刷怪笼方块。诊断：`/armedmobs capture`。

键名	默认值	作用	备注/推荐范围
<code>capture.enabled</code>	<code>true</code>	总开关。 <code>false</code> 时仍按楼分阵营、仍有驻军，但不建池、永不阻止刷新	
<code>capture.hudEnabled</code>	<code>true</code>	血条显示。纯客户端绘制（服务端只同步数字），关掉不可能改变胜负	
<code>capture.poolMin</code>	20	最小开局兵力（小城的地板）	4..200
<code>capture.poolMax</code>	100	最大开局兵力（大城的上限）	4..400
<code>capture.poolPerBuilding</code>	2	每栋楼给该阵营加多少人；4 栋 = 28 人，40 栋封顶 100	0..20
<code>capture.drainPerKill</code>	1	该城该阵营每死一人扣多少；1 = 池就是人头数（归零即「这些人真的死光了」）	1..10
<code>capture.playerKillsOnly</code>	<code>false</code>	只算玩家造成的死亡。默认 <code>false</code> = 两方自己也会打出胜负，玩家是加速器	
<code>capture.hudHideDelaySeconds</code>	8	离开城市后血条再停留几秒；只剩一方时立即隐藏	0..60

6. 常用调参配方

下面提到的每个键都在第 5 节存在；数值即「我会设成什么」。

6.1 让 AI 更凶 / 更快

- `ai.enabled = true` (确认总开关没关)。
- 反应: `ai.troop.reactionMinTicks = 2`、`ai.troop.reactionMaxTicks = 4`;
`ai.elite.reactionMinTicks = 2`、`ai.elite.reactionMaxTicks = 4`。
- 推进: `ai.troop.advanceCoverScale = 1.0`、`ai.elite.advanceCoverScale = 2.0`;
`ai.elite.coverSeekSpeedScale = 1.3`、`ai.elite.engageRangeScale = 0.5` (贴得更近)。
- 压制: `ai.troop.suppressChanceScale = 2.0`、`ai.troop.suppressTicksScale = 2.0`。
- 暴露火力: `ai.troop.exposedBurstShots = -1`、`ai.elite.exposedBurstShots = -1` (打空弹匣); `ai.troop.exposedBurstCooldownTicks = 0`、`ai.elite.exposedBurstCooldownTicks = 0` (不停顿); `ai.troop.warmupShotsWhenExposed = 0`、`ai.elite.warmupShotsWhenExposed = 0` (首发即稳态)。
- 精度: `accuracy.warmupShots = 0` (或 `accuracy.warmupMultiplier = 0.8`) , `[tiers.rifle] accuracy = 0.75` , `accuracy.profileRookieCap = 0.85`。
- 全局反应: `combat.reactionTicks = 6` (仅当 `ai.enabled = false` , 或某档缺省时才作为基准)。
- 手雷: `grenades.mob.carryChance = 0.7`、`grenades.mob.cooldownTicks = 120`、`grenades.mob.maxPerMob = 3`。
- 一击必杀的暴击来自枪本身: `guns.respectDeclaredFireModes = false` 保持全自动 (要「像莫辛那样」才设 `true`) 。
- 也可以只跑命令验证: `/armedmobs test watch 30 16` (看 `moved` 与 `shots`) 。

6.2 让 AI 更笨 / 更慢

- 一键变笨 (**revert to dumb**) : 保持 `ai.enabled = true` , 把 `[ai.scav]` 的每一个数抄进 `[ai.sniper]`、`[ai.troop]`、`[ai.elite]` (含 `holdPost=false`、`minHitChance=0.0`、`patienceTicks=0`、`coordination=false` , 以及 5ab 六键的哨兵值 `exposedBurstShots=0`、`exposedBurstCooldownTicks=-1`、`warmupShotsWhenExposed=-1`、`retreatHealthFraction=-1`、`hurtRetreatChance=-1`、`retreatHoldTicks=0`) 。
- 想要「加这功能之前」完全一致的手感, 再补: `ai.<tier>.reactionMinTicks = ai.<tier>.reactionMaxTicks = [combat] reactionTicks` (出厂 12) 。
- 只调慢不调档: `ai.troop.reactionMinTicks = 12`、`ai.troop.reactionMaxTicks = 24`;
`ai.troop.coverChance = 0.3`、`ai.elite.coverChance = 0.3`;
`ai.troop.suppressChanceScale = 0.0`。
- 精度: `ai.elite.accuracyScale = 0.8`、`accuracy.hardCeiling = 0.6`、
`accuracy.profileEliteCap = 0.6`。
- 血量/生存: `ai.troop.retreatHealthFraction = 0.9` (半血就退)、
`ai.troop.hurtRetreatChance = 1.0`、`combat.giveUpTicks = 60`。
- 枪械档: `[tiers.rifle] burstShots = 3`、`[tiers.rifle] accuracy = 0.4`、`[tiers.rifle] aimTicks = 30`。

- 完全关掉智力层: `ai.enabled = false` (四档全部不读, 回到只有全局键的旧行为)。

6.3 关掉门行为

- 只关关门: `ai.closeDoorsBehind = false`。
- 想把门相关数值也调到「几乎不关」: `ai.doorCloseDelayTicks = 1200`、`ai.doorCloseAllyRadius = 16.0` (只要有活体在附近就永不关)。
- 注意: `ai.closeDoorsBehind` 只影响关门; 单位仍会为了寻路开门 (由寻路决定)。日志标记见 `[doors]`。

6.4 关掉手雷

- 怪完全不扔: `grenades.mob.enabled = false`。
- 玩家与怪都不能用投掷物: `grenades.enabled = false`。
- 只禁止爆炸伤害而保留烟雾/闪光: `grenades.frag.fragmentDamage = 0.0`、`grenades.frag.fragmentCount = 0`、`grenades.he.blastPower = 0.0`, 并把 `grenades.blastDamagePerPower = 0.0`。
- 保留投掷但永不伤地形: 保持 `grenades.terrainDamage = false` (默认)。
- 想让雷更大而不更疼: `grenades.he.blastPower = 8.0` 而不要动 `grenades.blastDamagePerPower`。

6.5 调试 AI (看日志 / 看状态)

- 日志: `combat.logGunAi = true` (每帧状态切换、每个 TaCZ `ShootResult`、每个掩体决定); `spawn.logSpawnGate = true`; `client.logPoseWriters = true` (每帧 `[pose]` 写明姿势骨骼写入者); `ricochet.log = true`。
- 状态: `/armedmobs debug` (32 格内每只怪的 `GunBrain` 摘要 + 阵营/叛变/警戒/精度档/护甲等级/语音池/目标); `/armedmobs cover`; `/armedmobs gunpool [tier]`; `/armedmobs test mods`; `/armedmobs test sniper`; `/armedmobs client state`。
- 战斗闸门: `/armedmobs test fight [distance]` 搭场, `/armedmobs test watch [seconds] [distance] [pos]` 断言「移动 ≥ 1.0 格、开火 ≥ 3 发」, `/armedmobs test stall` 注入「每枪必失败」并断言看门狗触发 ≥ 1 次。三者都打印 `target=/los=/distance=` 便于区分「状态机坏了」和「测试点没视线」。
- 关掉日志: `combat.logGunAi = false`、`spawn.logSpawnGate = false`、`client.logPoseWriters = false`、`client.logGunMount = false`、`client.logHiddenBones = false`、`client.logRenderStats = false`、`client.logGlState = false`。

6.6 测试用召唤

- 城市内召唤 (穿门): `/armedmobs spawn scav`、`/armedmobs spawn sniper`、`/armedmobs spawn elite_pillager`。先把脚下标成城市: `/armedmobs city add test 64`。
- 任何地方召唤 (绕过门): 原版 `/summon tarkovscav:scav ~ ~ ~`; 只要 `spawn.gateCommandSpawns = false` (默认), `/summon` 与刷怪蛋都无视城市门。
- 想要「任何人都能被门挡住»: `spawn.gateCommandSpawns = true`。

- 免战斗快速造景: `/armedmobs city district` 就地拼城区; `/armedmobs city place <name>` 放单个导入的结构。
 - 检查为什么刷不出来: `/armedmobs city test` (读 `[spawngate]` 日志)。
-

7. 第三方内容接口

<!-- EXTENSION_SECTION_START -->

第三方（数据包 / 资源包 / 其它模组）能挂接的**全部**位置如下。数据包目录都相对 `src/main/resources/`（打包后为 jar 内 `/data/...`），资源包目录相对 `/assets/...`。

7.1 实体类型 id（在本模组基础上做内容时引用）

id	家族（语音）	智力档	说明
tarkovscav:scav	shared	SCAV	基础暴徒， <code>MobCategory.MONSTER</code>
tarkovscav:gunner_pillager	shared	SCAV	持枪掠夺者
tarkovscav:gunner_villager	shared	SCAV	持枪村民
tarkovscav:sniper_pillager	shared	SNIPER	<code>FOLLOW_RANGE</code> 更高
tarkovscav:sniper_villager	shared	SNIPER	村民半边的狙击手
tarkovscav:usec_villager	usec	TROOP	40 血、掷骰甲等
tarkovscav:bear_pillager	bear	TROOP	40 血、掷骰甲等
tarkovscav:elite_villager	elite	ELITE	40 血、最高精度档
tarkovscav:elite_pillager	elite	ELITE	同上
tarkovscav:grenade	—	—	唯一的投掷物实体（五种雷靠 NBT/物品区分， <code>MobCategory.MISC</code> ）

在数据包里引用即写 `tarkovscav:scav` 这样的 id（战利品表、刷新器、`entity_type` 标签等一律通用）。本模组**不导出**可被其它模组继承的公开实体类——要新增单位请参考 `AiProfile#tierFor` 的兜底逻辑：未识别的生物一律按 SCAV 处理。

7.2 本模组读取的实体类型标签（数据包覆盖路径）

标签	文件路径	作用	出厂值
#tarkovscav:faction_scav	data/tarkovscav/tags/entity_types/faction_scav.json	定义「scav 阵营」：同阵营互不主动开战（除非有人先动手）	tarkovscav:scav
#tarkovscav:faction_illager	data/tarkovscav/tags/entity_types/faction_illager.json	定义「掠夺者阵营」	原版 pillager/vindicator/evoker/illusioner/ravager + tarkovscav:gunner_pillager/sniper_pillager/bear_pillager/elite_pillager
#tarkovscav:faction_village	data/tarkovscav/tags/entity_types/faction_village.json	定义「村民阵营」	原版 villager/wandering_trader/iron_golem/snow_golem + tarkovscav:gunner_villager/sniper_villager/usec_villager/elite_villager
#tarkovscav:faction_village_hostile	data/tarkovscav/tags/entity_types/faction_village_hostile.json	村民阵营 见即攻击 的名单（由 <code>faction.villagersAttackMonsters</code> 开关控制）	#minecraft:raiders、 #minecraft:undead、spider、cave_spider、silverfish、endermite、creeper、slime、magma_cube、blaze、ghast、guardian、elder_guardian、shulker、vex、hoglin、zoglin
#tarkovscav:hard_target	data/tarkovscav/tags/entity_types/hard_target.json	「硬目标」：枪械伤害减免与跳弹规则作用的实体类型（ <code>[ricochet]</code> ）	minecraft:iron_golem

标签文件都是标准 Forge `replace: false` 列表; 数据包用同名文件 + `"replace": true` 或直接在 `values` 里追加即可。删除 shipped 文件会触发类内兜底表 (与文件内容一致), 行为不会更差。

7.3 结构: tarkovscav:city 结构标签 + structures/buildings/ 目录

本模组内置的城市 worldgen 结构放在标准数据包位置, 这就是第三方结构接入的样板:

文件	作用
<code>data/tarkovscav/worldgen/structure/city_small.json</code> 、 <code>city_a.json</code> 、 <code>city_b.json</code> 、 <code>city_c.json</code>	四个「整城」jigsaw 结构 (单模板)
<code>data/tarkovscav/worldgen/structure/city_district.json</code>	「街区」jigsaw 结构 (<code>size: 6</code> , <code>terrain_adaptation: none</code>)
<code>data/tarkovscav/worldgen/structure/city_strongpoint.json</code>	「18 栋据点」jigsaw 结构 (单模板, <code>terrain_adaptation: beard_thin</code> , <code>max_distance_from_center: 116</code>)。2026-09-23 修正: 原为 224, 而原版 codec 既限制区间 1..128, 又校验 <code>max_distance_from_center + 地形适配补偿 <= 128</code> , 违反时报 <code>Structure size including terrain adaptation must not exceed 128</code> , 导致客户端每次载入世界都报 <code>Failed to parse tarkovscav:worldgen/structure/city_strongpoint.json</code> 。补偿值由服务端 jar 反汇编 (<code>JigsawStructure\$1</code>) 确认: <code>none = 0</code> , <code>bury/beard_thin/beard_box = 12</code> , 所以 <code>beard_thin</code> 的最大合法值是 116。单件 (<code>size: 1</code>) 结构不做任何扩展, 这个字段不必覆盖 138 的占地; 现在由 <code>tools/selftest_datapack.js</code> 按真实规则钉死
<code>data/tarkovscav/worldgen/structure_set/city_variants.json</code> 、 <code>city_district.json</code> 、 <code>city_strongpoint.json</code>	结构集。四个城市结构集现在都用 <code>tarkovscav:wasteland_spread</code> 放置 (见下一行); 据点 <code>spacing 192 / separation 48</code> , 因为它的占地是 138×66
<code>data/tarkovscav/worldgen/structure_set/city_small.json</code> (以及上表三个)	密度旋钮: <code>"frequency": 0.25</code> —— 用户要求的「建筑刷率减少 75%」。javap 的 <code>StructurePlacement#isPlacementChunk</code> 证明判定顺序是「生物群系 → <code>if (frequency < 1.0F)</code> <code>shouldGenerate(seed, salt, x, z, frequency)</code> → 排斥区 → <code>isStructureChunk</code> 」, 所以 <code>frequency</code> 在放置逻辑之前生效, 同时作用于主世界和废土; 它保持网格/salt/separation 不变, 只把每个候选点的接受概率乘 0.25 (期望正好 25%)。另一条路是 <code>spacing</code> 加倍 (密度 ~ $1/\text{spacing}^2$), 但那会移动网格, 所以选了 <code>frequency</code> 。只影响新生成的区块: 存档里已经生成的城区位置不变

文件	作用
data/tarkovscav/dimension/urban_wasteland.json	废土维度：type: tarkovscav:urban_wasteland、generator: {type: minecraft:noise, biome_source: {type: minecraft:fixed, biome: tarkovscav:urban_wasteland}}, settings: tarkovscav:urban_wasteland}。注意：这里没有也不能有 structures 块——javap 的 net.minecraft.world.level.dimension.LevelStem 显示数据包用的 codec 只读 type 与 generator 两个字段，DimensionStructuresSettings 只在三个原版维度的 level.dat 路径上使用，所以写在数据包维度里的 structures 会被静默丢弃（实测：写进去时同一 512×512 采样区只有 2 个城区结构，正是主世界的稀疏间距）。废土的密度改由 tarkovscav:wasteland_spread 提供：isStructureChunk 检查该 chunk 生成器的 BiomeSource 是否为持有 tarkovscav:urban_wasteland 的 FixedBiomeSource，是则用 dense_spacing/dense_separation（4/1、5/2、16/5、192/48），否则用主世界的 spacing/separation，两条分支跑的是同一段原版随机散布算式（含带 salt 的 setLargeFeatureWithSalt）。因此主世界的生成与原版逐格一致（A/B 对照构建已验证）。spacing()/separation() 返回 dense 值，代价是主世界的 /locate structure 会走 dense 网格而找不到稀疏网格上的城区（这一现象此前已记录在 README 的 TODO 里）
data/tarkovscav/tags/entity_types/faction_village_hostile.json	「武装村民的敌对目标」标签（村庄防御逻辑读它）。2026-09-25 修正：原来引用了 #minecraft:undead，而 1.20.1 没有这个 entity_type 标签（客户端 jar 只有 13 个 entity_type 标签），TagLoader 会记 Couldn't load tag tarkovscav:faction_village_hostile as it is missing following references: #minecraft:undead 并把该部分静默置空——武装村民实际一个亡灵都不打。现在写成显式的 1.20.1 亡灵：#minecraft:skeletons + zombie/husk/drowned/zombie_villager/zombified_piglin/zombie_horse/phantom/wither。这是行为变更：村庄防御现在真的会打亡灵。tools/selftest_datapack.js 会把每个 #minecraft: entity_type 标签引用对照 1.20.1 的真实标签清单校验
data/tarkovscav/worldgen/template_pool/city_small/start.json、city_a b c/start.json	整城模板池
data/tarkovscav/worldgen/template_pool/city_strongpoint/start.json	据点模板池（单个 single_pool_element 指向 tarkovscav:city_strongpoint）
data/tarkovscav/worldgen/template_pool/city_district/start.json、street.json、building.json、decor.json	街区模板池（起始/街道/建筑/装饰）
data/tarkovscav/structures/city_small.nbt、city_a.nbt、city_b.nbt、city_c.nbt、gen_district.nbt、city_strongpoint.nbt	整城 / 据点的结构 NBT（city_strongpoint 由 tools/strongpoint-layout.json 生成，占地 138×34×66）。2026-09-26 起这些 NBT 里带方块实体：每栋楼 1-2 个 minecraft:spawner（只刷 tarkovscav:usec_villager / tarkovscav:bear_pillager，即 TROOP 档）和 1-3 个 minecraft:chest（LootTable: minecraft:chests/abandoned_mineshaft），用的是结构 NBT 的 blocks[].nbt 复合标签。刷怪笼的数值：SpawnCount 2 / SpawnRange 4 / Delay 60 / MinSpawnDelay 300 / MaxSpawnDelay 900 / RequiredPlayerRange 16 / MaxNearbyEntities 6；据点只放 4 个（spawner_budget）

文件	作用
<code>data/tarkovscav/structures/buildings/*.nbt</code>	街区部件: <code>street_tile_a</code> 、 <code>street_tile_b</code> 、 <code>building_a1</code> 、 <code>building_a2</code> 、 <code>building_b1</code> 、 <code>building_b2</code> 、 <code>gen_district_b1</code> 、 <code>gen_district_b2</code> 、 <code>decor_rubble_west</code> 、 <code>decor_rubble_east</code> 、 <code>wall_ring</code> (<code>wall_ring</code> 目前不在任何池里, 仅作资产)
<code>data/tarkovscav/tags/worldgen/structure/city.json</code>	# <code>tarkovscav:city</code> 结构标签: 列出 <code>city_small</code> 、 <code>city_a</code> 、 <code>city_b</code> 、 <code>city_c</code> 、 <code>city_district</code> 、 <code>city_strongpoint</code> 。 <code>spawn.cityStructureTags</code> 默认指向它
<code>data/tarkovscav/forged/biome_modifier/add_scavs.json</code>	生物群系刷新器 (<code>type: forged:add_spawns</code> , <code>biomes: #minecraft:is_overworld</code>), 九个实体的权重 <code>5/2/2/1/1/1/1/1/1</code> (合计 15; 原 <code>12/6/4/3/2/2/2/1/1</code> 合计 33)。权重是 <code>monster</code> 类别内的相对份额, 不是线性刷怪率
<code>data/forged/loot_modifiers/global_loot_modifiers.json</code>	本模组唯一的 global loot modifier 声明: <code>{"replace": false, "entries": ["tarkovscav:city_chest_extras"]}</code> 。第三方整合包可以往这里追加自己的条目 (<code>replace: false</code> 保证不会互相覆盖)
<code>data/tarkovscav/loot_modifiers/city_chest_extras.json</code>	城市箱子注入的条件: <code>tarkovscav:mod_loaded (modid: tacz) + forged:loot_table_id (loot_table_id: minecraft:chests/abandoned_mineshaft)</code> 。见 7.3.2

7.3.2 城区箱子的 TaCZ 注入 (global loot modifier, 2026-09-26)

城里的 `minecraft:chest` 用的是原版 `minecraft:chests/abandoned_mineshaft` (没有抄表, 所以别的 mod 往那张表的注入照样生效), TaCZ 的「弹药 + 改好的枪」由一个 Java 的 **Forge global loot modifier** 追加:

项	内容
注册 id	<code>tarkovscav:city_chest_extras</code> (<code>loot/CityChestLootModifier.java</code> , 注册进 <code>ForgeRegistries.Keys.GLOBAL_LOOT_MODIFIER_SERIALIZERS</code>)
JSON 条件	<code>tarkovscav:mod_loaded (modid: tacz) + forged:loot_table_id (minecraft:chests/abandoned_mineshaft)</code>
运行时双保险	<code>ModList.get().isLoaded("tacz")</code> : 没装 TaCZ 时什么都不做 (不报错、不产生空池)
位置限定	只有箱子 <code>LootContextParams.ORIGIN</code> 落在 CityGate 认定的城区里才注入 —— 真废矿保持纯原版。 <code>CityGate</code> 不写死维度, 所以主世界与 <code>tarkovscav:urban_wasteland</code> 的城区一视同仁
注入内容	1-3 个小弹药堆 (每堆 6-24 发, 口径取自 TaCZ 索引里“本模组允许的枪”所用弹药, 按物品自身最大堆叠截断); 5 % 概率一把枪 (档位权重: 步枪 50 / 霰弹 30 / 手枪 20), 枪由 现有 <code>GunPool.rollLoadout + GunPool.buildGun(loadout, random, owner)</code> 生成, 因此随机配件与满弹匣都是现成路径
日志	<code>[cityloot] city chest at <x,y,z> (<dimension>) received ...</code> , 每次真的注入都打一行
为什么不用 <code>forged:mod_loaded</code>	它是 crafting 条件 (<code>forged:conditions</code> 注册表), 而 GLM 的 <code>conditions</code> 由原版战利品 Gson 按 <code>"condition"</code> 在 <code>minecraft:loot_condition_type</code> 里解析, 那个注册表里 <code>forged</code> 只有 <code>forged:loot_table_id</code> 和 <code>forged:can_tool_perform_action (javap ForgeMod#registerLootData)</code> 。写了它整条 modifier 会被 Forge 丢弃并报 <code>Could not decode GlobalLootModifier</code> 。所以本模组自己实现了同语义的 <code>tarkovscav:mod_loaded</code>
闸门	<code>node tools/selftest_loot_modifier.js</code> (注册/声明/双保险/位置限定/数值/日志, 以及“任何 datapack JSON 里不许出现 <code>tacz: 物品 id</code> ”)

7.3.1 部署信标 `tarkovscav:deployment_beacon`（进入城市废土维度的主要方式）

项	内容
物品 id	<code>tarkovscav:deployment_beacon</code> （中文「部署信标」， <code>max_stack_size = 1</code> ，稀有度 <code>uncommon</code> ）
获取	工作台合成（有序配方 <code>data/tarkovscav/recipes/deployment_beacon.json</code> ）：竖排 <code>I / ITI / E</code> ， <code>I</code> = 铁锭 x3， <code>T</code> = 红石火把， <code>E</code> = 末影珍珠。创造模式物品栏也直接给
贴图	<code>assets/tarkovscav/textures/item/deployment_beacon.png</code> ，16×16，由 <code>node tools/make_beacon_texture.js</code> 程序化生成（无第三方素材）；模型 <code>models/item/deployment_beacon.json</code> （ <code>item/generated</code> ）
用法（主世界）	右键开始 40 tick（2 秒） 部署读条，结束时把玩家送进 <code>tarkovscav:urban_wasteland</code> 。读条期间播放 <code>minecraft:block.beacon.activate</code> 与烟雾/云粒子
用法（废土内）	右键开始同样的读条，结束时把玩家送回上次出发的主世界坐标（ <code>data/tarkovscav_retreat.dat</code> ，按玩家 UUID 记录）；没有记录时送回主世界共享出生点并且 明确告知
读条中断规则	受到任何伤害（ <code>LivingHurtEvent</code> ）、离开起点超过 4 格 （每 tick 检查）、提前松开右键、切换维度或掉线——每一种都有各自的提示
冷却 / 消耗	到达后 10 秒（200 tick） 冷却；物品永不消耗（是工具不是消耗品）
落地保护	目标列取玩家自己的 X/Z；Y 由该维度的 <code>MOTION_BLOCKING</code> 高度图决定，并向上找到两个不碰撞的格子；若落点下方没有可站立地面，则铺 5×5 石砖平台 并在角落放一支火把，同时提示玩家
拒绝条件	只在主世界（出发）或废土（返程）可用；非玩家来源、未知维度、冷却中、已有读条在跑——全部干净拒绝并给出提示（键在 <code>en_us / zh_cn</code> 都有）
共享代码	与 <code>/armedmobs dimension</code> 走同一套 <code>world/WastelandTravel.java</code> （同一个落地实现、同一条到达提示），所以命令路径与物品路径不可能各走各的
服务端可验证性	右键读条与落地平台 需要真实玩家 ，无头服务器上无法触发（控制台没有身体）；命令路径对控制台来源会干净拒绝

第三方城市模組的两种接入方式（都不需要改代码）：

1. 让本模组认别人的结构为城市：把对方的 `namespace:structure_id` 写进 `spawn.cityStructureIds`，或让 `spawn.cityStructureTags` 指向一个包含对方结构的标签。
2. 自己写一个标签文件（推荐，因为它可被多个整合包复用）：

```
data/<yourpack>/tags/worldgen/structure/city.json
{
  "replace": false,
  "values": [
    "tarkovscav:city_small",
    "yourmod:downtown"
  ]
}
```

然后把 `spawn.cityStructureTags` 设为 `["<yourpack>:city"]`（或把条目加进 `tarkovscav:city` 的 `values`，用 `"replace": false` 追加）。

往街区池里加自己的建筑部件：把自己的 NBT 放进

`data/<yourpack>/structures/buildings/<name>.nbt`，再往

`tarkovscav:city_district/building`（或 `street/decor`）池里加一个 `single_pool_element`：

```
{
  "weight": 2,
  "element": {
    "element_type": "minecraft:single_pool_element",
    "location": "yourpack:buildings/my_building",
    "processors": "minecraft:empty",
    "projection": "rigid"
  }
}
```



```
}
```

注意：`/armedmobs city district` 只读取本模组命名空间 (`tarkovscav:`) 的池元素；别人命名空间的部件会被跳过 (`location.startsWith("tarkovscav:")` 过滤)，所以要让命令也用到你的部件，请改 `tarkovscav:city_district/*` 池或提供你自己的一整套池。

7.4 自定义结构运行管线：`tarkovscav/city/*.nbt + city import|reload|place|structures`

这条管线服务于「不是 worldgen 结构」的城市：手工建筑、或用 `/place template` 粘贴的东西。

步骤	命令	产生的文件/日志
1. 在游戏里用结构方块把建筑保存为 <code>mycity</code>	—	<code><world>/generated/minecraft/structures/mycity.nbt</code>
2. 导入	<code>/armedmobs city import mycity</code>	复制到 <code><gameDir>/tarkovscav/city/mycity.nbt</code> ；校验失败会红字说明原因并写 <code>[city] import ... failed</code>
3. 重扫目录	<code>/armedmobs city reload</code>	重新注册 <code>tarkovscav/city/</code> 下全部 <code>*.nbt</code> (坏文件跳过并 WARN，不影响其它)
4. 放置	<code>/armedmobs city place mycity ~ ~ ~ 90 none</code>	记录该实例的包围盒； <code>CityGate</code> 把盒内的点视为城市区域；写 <code>[city] placed ...</code>
5. 查看	<code>/armedmobs city structures</code>	列出 jar 结构、运行时池 (含 <code>size/palette/blocks/entities</code>)、已放置实例
6. (可选) 设为显式城市盒	<code>/armedmobs city add mycity 128</code>	写入 <code>spawn.cityRegions</code> (格式见第 5.1 节)

cityRegions 字符串格式 (也是手写进 toml 的格式)：

```
[<name>]|<dimension>|<x1> <y1> <z1>|<x2> <y2> <z2>
minecraft:overworld|-120 40 -80|120 120 80
downtown|minecraft:overworld|-120 40 -80|120 120 80
```

- 三段式 (无名字) → 名字解析为 `region`；四段式 → 第一段为名字。
- 坐标可用空格或逗号分隔；两个角点顺序无关 (解析时取 min/max)。
- 名字匹配大小写不敏感；`/armedmobs city add` 同名的旧条目会被替换。

运行时结构的校验上限 (导入/重载时执行)：单轴 ≤ 512 格 (> 256 只 WARN)、方块数 ≥ 1 且 ≤ 250000、必须有非空 palette、每个 block 的 `state` 必须指向存在的 palette 项、文件必须是压缩 NBT。运行时 `tarkovscav/city/*.nbt` 不会变成 worldgen 结构 (没有 `structure json/set/biome modifier`)，自然生成需要发布为数据包并重启。

7.5 自定义语音包 (`voice_clips.txt` / `sounds.json`)

运行时的语音清单是 `assets/tarkovscav/voice_clips.txt`，一行一个 **sound event 名**，`#` 开头为注释；`ModSounds` 在类构造时读取它，并用 `<family>_<category>` 规则把事件名分组 (去掉末尾的 `_<数字>`)。

- **家族 (family)**：`usec`、`bear`、`elite` (`ModSounds.FAMILIES`)；另外 `shared` 是「不属于任何家族的怪」所用池，且是家族没有自有 clip 时的回退池。

- **类别（category）**：AI 实际会请求的六个类别是 **contact**、**chatter**、**idle**、**grenade**、**mark**、**death**（`VoicePools.pool(mob, shared, category)` 的调用点）。另外 **grenade_land** 是手雷落地效果池（`ModSounds.IMPACT_POOL`）。
- 命名即池名：**voice.usec_contact_3** → 池 **usec_contact**；因此加一条语音 = 在清单里加一行 + 在 **sounds.json** 里加一条 + 放一个 **ogg**。
- **test sound** 认得的池名：**all**、**idle**、**chatter**、**contact**、**taunt**、**grenade**、**mark**、**death**、三个家族名、全部家族池名，以及单个 clip 短名。

最小可用示例（第三方资源包，**replace: false** 追加）：

```
# 1) 资源包里的 assets/<yourpack>/... 不会进本模组的清单；清单固定读 tarkovscav 命名空间。
# 正确做法是覆盖 assets/tarkovscav/voice_clips.txt（或提供同路径的更高优先级资源包）：
assets/tarkovscav/voice_clips.txt
# 追加你自己的池（会与同池的内置 clip 合并）
voice.elite_contact_9
voice.elite_contact_10

# 2) 事件必须在 assets/tarkovscav/sounds.json 里定义（否则是静默失败）：
assets/tarkovscav/sounds.json
{
  "voice.elite_contact_9": {
    "category": "hostile",
    "subtitle": "subtitles.tarkovscav.elite_contact_9",
    "sounds": [ { "name": "tarkovscav:voice/elite_contact_9", "stream": false } ]
  }
}

# 3) ogg 文件放在：
assets/tarkovscav/sounds/voice/elite_contact_9.ogg # 必须是单声道 44.1 kHz

# 4) 字幕键（可选但本模组自带的两份语言文件都有）：
assets/tarkovscav/lang/en_us.json / zh_cn.json
{ "subtitles.tarkovscav.elite_contact_9": "..." }
```

tools/voice_plan.js 使用的 **tools/voice_pools.json** 是构建期的池分配表（生成器用它挑选素材），没有打包进 jar；运行时唯一被读的清单是 **assets/tarkovscav/voice_clips.txt**（若该文件缺失，**ModSounds** 会 WARN 并只保留手写声明的基础 clip）。

7.6 枪械池 / TaCZ 集成键

「哪把枪能被发给怪」完全由 **[guns]** 与 **[tiers.<gun>]** 决定，不需要改代码；TaCZ 枪包的 **type** 字段自动喂给枪械档。

键	对第三方枪包的意义
guns.gunBlacklist / guns.gunWhitelist	精确到 gun id（ namespace:gun_id ）的排除/白名单
guns.excludedGunTypes	按 TaCZ gun type 整类排除（默认 rpg ）
guns.pistolClipTypes	哪些 type 用 *_pistol 动画（默认 pistol ）
guns.excludeScriptedGuns / guns.trustedScriptNamespaces / guns.scriptedGunRescanTicks	Lua 脚本枪的控制（默认只信任 tacz 命名空间）
guns.respectDeclaredFireModes	是否尊重枪数据文件声明的开火模式（默认强制全自动）

键	对第三方枪包的意义
<code>mods.*</code> （5 个键）	随机配件池：AttachmentType 槽位为 SCOPE、MUZZLE、STOCK、GRIP、LASER、EXTENDED_MAG；候选来自 TaCZ 自己的配件索引 (TimelessAPI.getAllCommonAttachmentIndex())，含 id + getType())，是否合法由 TaCZ 的 IGun.allowAttachmentType / allowAttachment 回答，本模组不自己判定
<code>[tiers.<gun>]</code> （7 键 × 4 档）	该枪械档的血量/护甲/精度/瞄准/点射长度与间隔

- 观察实际池：`/armedmobs gunpool rifle`（会打印每把枪与其 type）与 `/armedmobs test mods`（会打印 TaCZ 配件索引是否为空、每种槽位可用数量）。
- **TaCZ 自家枪包/配件包的文件布局与 id 规则**（例如配件定义放在哪个目录、attachment id 的命名约定）（**未核对**）——本文只确证本模组读取 TaCZ 公开 API 的方式（见 `gun/GunAttachments.java`、`gun/GunPool.java`）。请以 TaCZ 官方文档为准。

7.7 其它扩展点

扩展点	位置	说明
生物群系刷新权重 / 生物群系	<code>data/tarkovscav/forge/biome_modifier/add_s cavs.json</code>	<code>forge:add_spawns</code> ；数据包覆盖即可改权重、minCount/maxCount、目标生物群系标签
武器放置台战利品 / 配方	<code>data/tarkovscav/loot_tables/blocks/weapon_ rack.json</code> 、 <code>creative_weapon_rack.json</code> 、 <code>data/tarkovscav/recipes/weapon_rack.json</code>	普通台与军械台（创造）是两个不同方块/方块实体，不是 NBT 标志
手雷配方	<code>data/tarkovscav/recipes/{frag,flash,flash_ grenade_short,he,smoke}_grenade.json</code>	五种投掷物的合成
模型骨骼 / 锚点	<code>client.gunAnchorBone</code> 、 <code>client.gunOffhandAnchorBone</code> 、 <code>client.hiddenBones</code> 、 <code>client.headAccessories.*</code>	做 rig 时改这些键即可挂枪/隐藏配件；受保护骨骼（身体与服装层）无法隐藏
姿势归属 / Molang	<code>client.poseSource</code> 、 <code>client.molangVariables</code> 、 <code>client.torsoYawShare</code>	做动画 rig 时决定作者 clip 与代码谁写骨骼、喂哪些 ysm.*/query.* 符号
硬目标集合	<code>#tarkovscav:hard_target</code>	把任意实体类型加入即受 [ricochet] 规则影响

<!-- EXTENSION_SECTION_END -->

8. 数据 / 文件位置与日志标记

8.1 文件位置

路径	内容
config/tarkovscav-common.toml	全部配置键（第 4 节）。Forge COMMON 类型； /armedmobs client reload 可从磁盘重读
<gameDir>/tarkovscav/city/*.nbt	city import 复制进来的运行时结构池； city reload 扫描此目录
<world>/generated/minecraft/structures/<name>.nbt	结构方块保存的输出， city import 的输入
data/tarkovscav/structures/buildings/*.nbt	jar 内置街区部件
data/tarkovscav/structures/{city_small,city_a,city_b,city_c,gen_district}.nbt	jar 内置整城
data/tarkovscav/worldgen/{structure,structure_set,template_pool}/**	城市 worldgen 定义
data/tarkovscav/loot_modifiers/city_chest_extras.json、 data/forge/loot_modifiers/global_loot_modifiers.json	城区箱子的 TaCZ 注入（第 7.3.2 节）
data/tarkovscav/tags/{entity_types,worldgen}/**	实体标签与结构标签（第 7.2 / 7.3 节）
assets/tarkovscav/voice_clips.txt	语音事件清单（一行一个）
assets/tarkovscav/sounds.json	每个事件的文件引用与字幕键
assets/tarkovscav/sounds/voice/*.ogg、 assets/tarkovscav/sounds/effect/grenade_land.ogg	音频文件（要求单声道 44.1 kHz）
assets/tarkovscav/lang/en_us.json、zh_cn.json	命令反馈与字幕文本

8.2 日志标记（测试者按标记搜索）

标记	来源	关键字段
[spawn gate]	CitySpawnEvents、ModCommands (city test / spawn)	ACCEPT/REJECT 、位置、原因、区块结构诊断
[gunai]	GunBrain	状态切换、 dist= 、 los= 、 cover= 、 health= 、 shots= ； exposed= （本 tick 目标是否暴露）、 retreatHold= （剩余蹲守 tick）；无进展 WARN 里带 doors: open= pass= closes= pending= ； shoot -> <TaCZ ShootResult>
[doors]	DoorBehavior	门的开/关决策
[sniper]	SniperBehavior、ModCommands test sniper	占位、换位原因、重新锁定、狙击手摘要
[grenade]	GrenadeThrowGoal、GrenadeBlast、GrenadeEvents、GrenadeResupplyGoal	投掷/扣住原因（弧被挡、落点有队友）、爆炸、烟雾云、补雷
[mods]	GunAttachments、ModCommands test mods	配件池可用数、装上的配件、TaCZ 索引为空、非法组合拒绝
[rack]	WeaponRackBlockEntity、WeaponRackTaker、WeaponRackArmament、ModCommands test rack	取用与转化、吸收掉落物、创造台不消耗、不支持物品 WARN
[voice]	MobVoice、ModSounds、Config	播放的行/池/音高/音量、清单缺失、 pitchMin > pitchMax 、非法 familyVolume
[city]	CityStructures、CityDistrictAssembler、ModCommands、TarkovScav	导入/加载/放置/区域新增、池读取、地基深度不一致 WARN 、缺件 WARN
[cityloot]	CityChestLootModifier	每次城区箱子真的收到 TaCZ 追加物时一行：位置、维度、收到了什么（<数量>x<物品>）
[fillwater]	WaterCleanup、ModCommands fillwater	维度、目标方块、包围盒、扫描方块数、替换数、含水方块跳过数、落在扫描边界上的水数（会流回来的那批）

标记	来源	关键字段
[capture]	CityCapture、ModCommands capture	池建立、每次扣减、 captured by <胜者> (<败者> strength exhausted) (只写一次)、驻军被拦截 (refused by the capture veto)。废土维度永不出现
[pose]	PoseWriters、RigSupport	每帧每根姿势骨骼的写入者、双写 WARN
[test]	FightHarness、ModCommands test fight/watch/stall	WATCH PASS/FAIL ... moved= shots= dummyDamage= stalls= state= 、 STALL PASS/FAIL ... escapes= 、 target=/los= 上下文
[cover]	ModCommands cover	候选点数、隐蔽数、最佳掩体
[debug]	ModCommands debug	每只怪的完整状态机与阵营摘要
[client]	ClientCommands	各客户端命令写入了什么值
[sound]	ModCommands test sound	逐条播放的事件、序号、音高、音高带

9. 未核对项与源码/README 不一致

9.1 (未核对)

1. `killFeed.mode` 除出厂值 `"involved"` 之外的全部合法取值——本次只读了 `Config` 中的默认串与 `ClientCommands` 对该键的读取，未逐个核对解析函数的取值集合。请以 README §5u 为准。
2. TaCZ 枪包 / 配件包自身的文件布局与 `id` 命名规则（本模组只通过 TaCZ 公开 API 读取索引）。本文只确证了本模组读取的部分。
3. 语音池 `tools/voice_pools.json` 中的中文分类名（该文件在仓库里以非 UTF-8 形式保存，读取呈乱码），因此本文不引用其中的分类名，只引用代码真正接受的 `<family>_<category>` 六类。
4. `client.startMode` 的合法取值集合（源码中默认串为 `"immediate"`，本次未追进解析函数）。

9.2 撰写本文时发现并已修复的文档漂移 (2026-09-25)

下面 7 条是逐条核对源码时发现的真实漂移，已全部修好——README、配置注释与源码现在一致，所以你可以按本文的默认值表照抄：

1. `client.modelLayering`: `Config#singleControllerMode()` 的 javadoc 曾写「`single`, the default」，与常量 `DEFAULT_MODEL_LAYERING = "upperLower"` 矛盾 → javadoc 已改为「`upperLower` 是出厂默认」，并写明 `single` 会冻结腿部（枪械 clip 没有腿部轨道）。
2. README `[spawn]` 表的 `cityStructureIds` 默认值过时：曾写 `["tarkovscav:city_small"]` → 已改为源码 `defaultCityStructureIds()` 的四个预设（`city_small`、`city_a`、`city_b`、`city_c`）。
3. README `pistolClipTypes` 默认值过时：曾写 `["pistol", "smg"]` → 已改为源码的 `["pistol"]`（想给 SMG 单手姿势就把 `smg` 加进来）；README `[guns]` 表也补上了 `excludeScriptedGuns`、`trustedScriptNamespaces`、`scriptedGunRescanTicks`、`respectDeclaredFireModes` 四行。
4. `foundationDepth` 的配置段：README §7 曾写 `[city] foundationDepth` → 已改为 `[spawn] foundationDepth`（源码注册在 `[spawn]`），README `[spawn]` 表补了该行，`CityDistrictAssembler` 的告警文案从 `city.foundationDepth` 改为 `spawn.foundationDepth`，`安装说明.txt` 同步。
5. `spawn` 命令的「Unknown mob」提示串曾漏写合法值 `sniper_pillager` → 已补上；现在提示串与本文第 1 节的九个 id 完全一致。
6. `gunnerVillagerGunRotation` 的注释与常量不一致：常量与写进 toml 的默认值是 `["5", "0", "0"]`（本文第 5.15 节即按此写），但 `Config` 的 javadoc 与 toml 注释仍按历史的 `[-40, 0, 0]` 与 `-220 / -190 / -150` 描述 → 两处都已改写为「5 是 2026-09-25 起的发布基线，-40 是历史值」，并指向 README §5j 的四档表（`-185 / -130 / -128 / -128`）。`gunnerVillagerIdleGunRotation` 的注释同样从历史的 `-70 / -220` 更正为出厂的 `-2`（闲置 -130）。
7. `ModSounds` 的 javadoc 计数措辞：曾写「Twenty-seven mono Vorbis clips」→ 已改为「121 个 sound event（基础池 + `voice_clips.txt` 清单）」，与 `voice_levels.json` 的行数一致。

这 7 条修的都是文档与注释，不是行为：5、`upperLower`、`spawn.foundationDepth` 本来就是编译进 jar 的出厂值，本次只是让文字追上它们。本文第 5 节的默认值表全部取自常量，因此不受这次修改影响。